A Mini Project Report On

TAG AND TRAIL

Submitted in Partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

In

Department of Computer Science and Engineering

By

Mupparaju Rohith	23241A05H5
P.Akshath Varma	23241A05J3
Sidhu Namburi	23241A05J8
Vyalla Adhivenkataramana Reddy	24245A0521

Under the Esteemed guidance of

Bh.Prashanthi

Assistant Professor

Department of Computer Science and Engineering

GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Bachupalli,Kukatpally,Hyderabad,Telengana,India,500090

2025-2026

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

CERTIFICATE

This is to certify that the Mini Project entitled "**Tag And Trail**" is submitted by **Mupparaju Rohith(23241A05H5), Pinnamraju Akshath Varma(23241A05J3) , Sidhu Namburi(2324 1A05J8) ,Vyalla Adhivenkataramana Reddy(24245A0521)** Partial fulfillment of the requir ements for the award of the degree of BACHELOR OF TECHNOLOGY in Computer Science and Engineering during the academic year **2025-2026.**

INTERNAL GUIDE	HEAD OF THE DEPARTMENT
Bh.Prashanthi	**Dr. B. SANKARA BABU**
Assistant Professor	**Professor & HoD**

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

Many people helped us directly and indirectly to complete our project successfully. We would like to take this opportunity to thank one and all. First, we wish to express our deep gratitude to our internal guide **Bh.Prashanthi**, **Assistant Professor**, Department of CSE for his support in the completion of our project report. We wish to express our honest and sincere thanks to **V Jyothi and K Adi Lakshmi** for coordinating in conducting the project reviews. We express our gratitude to **Dr. B. Sankara Babu, HOD**, department of CSE for providing resources, and to the principal **Dr. J. Praveen** for providing the facilities to complete our Mini Project**.** We would like to thank all our faculty and friends for their help and constructive criticism during the project completion phase. Finally, we are very much indebted to our parents for their moral support and encouragement to achieve goals.

Mupparaju Rohith **23241A05H5**
Pinnamraju Akshath Varma **23241A05J3**
Sidhu Namburi **23241A05J8**
Vyalla Adhivenkataramana Reddy **24245A0521**

DECLARATION

We hereby declare that the Mini Project entitled "**Tag And Trail** " is the work done during the period from 12th Dec 2025 to 25th April 2026 and is submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from **Gokaraju Rangaraju Institute of Engineering and Technology.** The results embodied in this project have not been submitted to any other university or Institution for the award of any degree or diploma.

Mupparaju Rohith	**23241A05H5**
Pinnamraju Akshath Varma	**23241A05J3**
Sidhu Namburi	**23241A05J8**
Vyalla Adhivenkataramana Reddy	**24245A0521**

	LIST OF FIGURES	
Fig No	**Fig Title**	**Page No**
4.3.1	System Architecture	19
4.3.2	Class Diagram	21
4.3.3	Use Case Diagram	23
4.3.4	Sequence Diagram	25
5.1.1	Project Folder Structure	31
5.1.2	App.tsx	32
5.2.1	Login Screen	44
5.2.2	Menu Section	44
5.2.3	DashBoard Screen	45
5.2.4	Upload Section	45
5.2.5	System Logs Section	46
5.2.6	Stats Screen	46
5.2.7	Uploaded Pdf Section	47
5.2.8	QR code to install the Tag And Trail App	47

	LIST OF TABLES	
Table No	**Table Name**	**Page No**
5.3.1	Authentication Module TestCases	48
5.3.2	Document / Link Input Module TestCases	48
5.3.3	Processing & Tag Generation TestCases	48
5.3.4	Classification Module TestCases	49
5.3.5	5. Search & Filter TestCases	49
5.3.6	6. Safety Check Module TestCases	50

ABSTRACT

The project showcases an AI-based system to gather, analyse and index links and PDF documents distributed on digital platforms. As the amount of online materials shared via messaging applications, email and social media platforms grow at a fast rate, users find it hard to handle and access valuable information. The proposed system addresses this issue by offering a smart and centralized content management solution to allow efficient storage and access to the content.

The system manipulates links and PDF files via a machine learning pipeline in the cloud where content is extracted, cleaned, and structured. Machine learning algorithms are used to classify documents, create summaries and attach tags to documents, and the user can enhance this by retraining. Every user has his or her local storage storage where the privacy and offline access are guaranteed. The system provides a personalized and searchable body of knowledge by integrating machine learning, artificial intelligence, and cloud computing.

CHAPTER-1

INTRODUCTION

The manner in which individuals relate to information in the contemporary digital era has transformed tremendously. As the use of smartphones and access to the internet increases, users are being subjected to huge volumes of data on a daily basis. This information is presented in different formats, including web links, PDF files, images and videos, and is disseminated on numerous platforms like WhatsApp, Instagram, LinkedIn, email, and other social media apps. Although this has ensured that communication has been quicker and more effective, it has also posed new challenges regarding how to handle and arrange this constantly increasing amount of content.Users nowadays are presented with various sources of information during a day. As an example, a student may get lecture notes sent through WhatsApp, a professional may get the reports sent to an email, and a job seeker may surf the opportunities posted on LinkedIn. All this is useful in various situations, and it is usually dispersed between platforms. There is no centralized system to manage this content, thus, users are left with manual ways of remembering like bookmarking, saving files in local drives or even by remembering where they viewed something. This is challenging and ineffective in the long run.

Among the most typical instances of this problem, it is possible to mention such messaging apps as WhatsApp. When users get several links, they are typically displayed as a category under the general “Links” section. This feature is basic in terms of organizing but does not offer more insight or classification of the content. The connections are shown in the form of simple URLs without descriptive or abstractive features. Consequently, users are not able to determine the nature of the content easily. The connection may be to an educational article, a news announcement, a technical blog, or a job opportunity, but is not made clear unless the user clicks on it.

The more the links the more time you spend on this process. Users are regularly bombarded with dozens of links in a day, and it is not feasible to read them out one at a time. Because of this, a lot of links are disregarded or overlooked even when they have valuable information. In the long term, it results in loss of what might be valuable resources. The situation is even more difficult due to the absence of a smart mechanism that would be able to classify and generalize links.The same is the case with PDF documents. The most common PDF files that are downloaded or received by users are study materials, reports, manuals, resumes, and research

papers. Such files are typically stored in machines that have faulty or ambiguous file names. As an example, files can be named such as document1.pdf or notes final.pdf which do not give any meaningful information on what is contained in them. The more files are stored the more the user will find it hard to remember what each file has and where it is stored.

In most scenarios, users tend to save documents in any folder without organizing them. This causes confusion when they are later attempting to get a particular file. The user might even have to open the file and go through a number of pages before realizing how it is relevant to him or her. This will lead to the wastage of time and effort. Moreover, users can download or store copies of files just because they are unable to find the original one, which also results in an unwarranted use of storage.The other critical issue of this issue is that there is no fast learning. In today's fast-paced environment, users do not always have the time to read lengthy articles or documents in detail. Rather, they enjoy the fast summaries or highlights to enable them determine whether the content is useful. Nonetheless, the majority of the current systems are more oriented at information storage, but do not assist users interpret it.

It is not just a matter of content storage but also of meaningful and available content. Users require systems that are able to save data more than what the data save systems do. They need devices that have the capability to scan content, isolate main points and bring them out in a simplified and structured format. Even properly stored information may turn out to be a challenge without such features.

The necessity of proper content management is even greater when one takes into consideration the level of dependence that people have developed on the digital information. Students depend on common links and PDFs to study. Digital documents are utilized by professionals when communicating, researching, and making decisions. Online content is used by individuals to keep them informed and updated on the current events. In any of these situations, rapid access and comprehension of information is an important factor.The traditional ways of handling information are no longer adequate as the amount of digital content keeps on increasing. The complexity and size of current data cannot be managed with manual organization, simple storage systems and basic search capabilities. More intelligent systems that can be used to automate the process of information organization and retrieval are required.

A smarter, easier to use solution is needed to overcome these challenges. With the help of such a system, it must be able to automatically analyze content and determine the main information

and sort it into useful categories. It must also include such features as summarization and tagging, which assist users to grasp the content quickly without necessarily reading it in details. Through this, the system would greatly save time and effort that would otherwise be spent to control digital information. The smart content management system can change the interaction of users with their data. The system can transform the links and documents instead of merely storing them into knowledge. To illustrate, an example of a shared link in a chat room can be automatically analyzed, categorized and tagged according to the content. Likewise, a PDF document can be manipulated to get the main points and create a summary. This ensures the ease with which information is located and used by the users as and when required.Moreover, this type of system can assist users to create their own knowledge base. The system will be able to offer a more personal experience by sorting the content according to the preferences and usage patterns of the users. The information can be easily searched by the users in terms of key words or tags and be retrieved easily. This not only enhances efficiency, but also contributes to the overall user experience.

Moreover, the system can also be enhanced by incorporating machine learning and natural language processing algorithms. These technologies allow the system to understand the context and meaning of content, rather than just processing it as plain text. The system will be able to learn as time goes by and become more accurate in classification and tagging.To sum up, the growing amount of digital content has necessitated the need to come up with smarter means of handling information. The existing ways of managing links and documents are not adequate to satisfy the modern users. The intelligent system that is able to automate the organization of the content, enhance accessibility, and better understanding is clearly needed. Seeking to overcome these challenges, the proposed system will offer a viable and effective solution to the digital information management in the contemporary rapidly changing world.

CHAPTER-2

LITERATURE SURVEY

Genetic-Algorithms Matrix-Factorization-Based Recommender System Using Tags (2025) The paper suggests the use of a tag-based recommender system (ETagMF) that integrates matrix factorization with genetic algorithms. Genetic algorithm is the one that is applied to the prediction of rating and the operations are selection, crossover and mutation. The model is tested by RMSE and is found to be more accurate and faster than the traditional MF and EMF techniques. The system however has a limitation of the movie recommendation datasets and must have structured domain specific data. It fails to facilitate document or link processing, has no content extraction features and does not include a centralized or personalized knowledge base and is not suited in real world multi-source data applications.

Extracting Sentences Embeddings from Pretrained Transformer Models (2025) In this study, the authors are interested in extracting sentence-level embeddings using pretrained transformers (such as BERT) without any fine-tuning. It analyses various pooling methods of tokens including mean pooling, weighted pooling and combinations of layers. The approaches are evaluated based on Semantic Textual Similarity (STS), the performance in clustering and the accuracy of classification. Although the quality of representation is impressive, the work is restricted to sentence embeddings, and does not cover document-level processing, summarization or tagging. It is also not integrated with storage or content management systems and is not oriented towards real time deployment.

Web Crawling Algorithm Fusing TF-IDF and Word2Vec Feature Extraction (2025) The paper provides a topic-based web crawling algorithm, which integrates both TF-IDF and Word2Vec algorithms to extract similar features based on semantics. The system enhances filtering of content relevancy, eliminates redundancy and organizes extracted web data. It is measured in terms of precision, recall, F1-score, retrieval time and coverage. It however, does not support document summarization or tagging, PDF processing, personalization or feedback features. Also, it does not support scalable, cloud-based or multi-user architecture.

PDF Malware Detection: Towards Machine Learning Modeling with Explainability Analysis (2024) This article deals with the detection of malicious PDFs using machine learning (ML) models, including Random Forest, SVM, AdaBoost, KNN, Gradient Boosting, and DNN. They are followed by feature extraction methods and the best results are obtained with the help of Random Forest in terms of accuracy measures. Although useful in terms of security, the system can only be used to detect malware. It uses a lot of handcrafted features, does not have the capability to be deployed in real-time, and has no document management or user-centered knowledge system integration.

Exploring the Landscape of Automatic Text Summarization: A Comprehensive Survey (2023) The paper represents a comprehensive overview of the methods of automatic text summarization, such as extractive and abstractive methods and hybrid methods. It discusses different methods including rule-based, statistical, machine learning, deep learning, and hybrid. Such evaluation measures as ROUGE and BLEU are mentioned. But it is a mere survey study, and does not suggest a practical implementation. It is restricted to the summarization and it is not related to the management of documents, tagging, retrieval, personalization, and real-time integration with clouds.

The study by A Retrieval-Augmented Generation Based Large Language Model Benchmarked on a Novel Dataset (2023) presents a Retrieval-Augmented Generation (RAG) framework, which consists of document retrieval and large language models. It has a modular design that enables it to easily integrate retrievers and generators. The semantic similarity is evaluated on the basis of cosine similarity scores. Nevertheless, it fails to employ typical summarization metrics, evaluators of storage and retrieval efficiency, and does not emphasize user-related document organization or personal knowledge systems.

SemGloVe: Semantic Co-occurrences for GloVe with BERT (2022) The paper suggests SemGloVe, a post-hoc extension of GloVe, which includes semantic co-occurrence data obtained with BERT. It boosts word embeddings through the use of contextual information with attention mechanisms and masked prediction. The model enhances the similarity of words and analogy. Nevertheless, it remains to generate static embeddings and not fully contextual. It is only restricted to word-level operations, is computationally expensive and cannot do document-level operations, summarization or real-time large-scale operations.

Unsupervised Keyphrase Extraction with Interpretable Neural Networks (2022) This study introduces the INSIGHT unsupervised keyphrase extraction system that can find meaningful phrases according to how much they indicate the topic of the document using interpretable neural networks. In contrast to the traditional methods based on the heuristic rules or graph-based ranking, it applies the model-based importance scoring and attains the state of the art performance on various datasets. Although it is effective in getting keyphrases that are meaningful, the method exclusively concentrates on the importance of phrases and topic prediction. It does not deal with integration to end-to-end systems like document storage, retrieval or real-time tagging pipelines, or multi-source inputs like web links or user-generated knowledge bases.

PatternRank: This paper presents PatternRank, an unsupervised keyphrase extraction system that uses pretrained language models along with part-of-speech (POS) patterns to extract and rank keyphrases. It is more accurate, recalls better and F1-score than earlier methods and can be customized to any domain by customizable POS patterns. Although it can be very useful in extracting keywords, the system can only extract key phrases of individual documents but lacks the wider scope of activities like document classification, storage and semantic retrieval. It is also not integrated with real-time processing pipelines, or multi-format content sources such as links and PDFs.

LongKey: Keyphrase Extraction for Long Documents (2024) This paper introduces LongKey, a keyphrase extractor that is optimized to extract keyphrases in long documents by using encoder-based language models and max-pooling embeddings to extract long-contextual information. It has better performance than current techniques on various datasets, particularly on long-form content; traditional models can fail because of input length constraints. Although useful in processing large text inputs, such as PDFs and articles, the system only covers the keyphrase extraction aspect and does not consider other features such as document classification, tagging pipelines or other integration with user-friendly knowledge management systems. Also, it does not take into account real time deployment or multi source content ingestion like web links and messaging platforms.

CHAPTER-3

SYSTEM REQUIREMENTS

3.1 Software Requirements

Operating System: Windows / Linux / macOS

Libraries & Frameworks: TensorFlow / PyTorch (Deep learning models), scikit-learn (Random Forest classifier), spaCy (Part-of-Speech tagging), Sentence Transformers (SBERT embeddings), BeautifulSoup (Web scraping), PyPDF2 (PDF processing), React Native (Frontend)

APIs & Services: Twilio API (WhatsApp integration), VirusTotal API (Malware detection), Hugging Face, MongoDB

3.2 Hardware Requirements

Processor: Intel Core i5 / AMD Ryzen 5 or higher

RAM: Minimum 8 GB (16 GB recommended for model training)

Storage: At least 20 GB free disk space for datasets, models, and documents

Internet Connectivity: Required for API communication (Twilio, VirusTotal)

3.3 Methodology & Data Set

Data Ingestion: Users post links or documents with a WhatsApp bot connected via the Twilio API. Webhook requests receive media files or URLs to the system.

Content Identification and Standardization: The system identifies the type of input (URL or PDF and so on) and the type of file. All formats that are not supported (images, DOCX, TXT, and others) are changed into PDF to be processed identically.

Malicious Content Detection: URLs are examined through a 1D CNN + LSTM deep learning network that is trained to spot a malicious link. A random forest classifier in the form of PDFs is used to assess the documents. The VirusTotal API is used to check suspicious content to provide another security verification.

Accessibility Checking: The system identifies the content in terms of being public or restricted. Authenticated or restricted access URLs are highlighted, and password-protected PDFs indicated.

Metadata Extraction: The webpages or PDF documents are analyzed to extract important textual data (title, description, initial content, etc.) that can be used.

NLP Processing: spaCy is used to perform part-of-speech tagging to filter meaningful keywords and Sentence-BERT embeddings are obtained to learn semantic associations among words.

Tag Generation & Optimization: The scoring of candidate keywords is based on penalty-based scoring and Maximal Marginal Relevance (MMR) is used to ensure that the tags are diverse and relevant.

Description of Dataset: URL Classification Dataset

The machine learning models in this project were trained and tested on a URL Classification dataset, which is a binary classification task to differentiate among the various types of links provided by web pages.

Dataset Overview

In this dataset, there are 747,886 entries of the URLs each marked as supervised learning. It is designed in a way that it helps in classification models which may automatically recognize patterns in web addresses.

The records of the dataset are composed of:

URL: URL (input feature)

Name: Classification target (binary).

0 → Legitimate / Safe URL.

1 0 Malicious / Suspicious URL

The dataset is big-data, which is why it is appropriate to train high-performance machine learning and deep-learning models. Its size enables models to acquire various trends in URL designs, enhancing its capacity to generalize to unobserved statistics.

Dataset Characteristics

Total samples: 747,886

Features: 1 main feature (URL text)

Variable under investigation: Binary (0 or 1)

Type of task: Classification (Supervised Learning).

The dataset is text-based feature learning, in which the models learn lexical and structural patterns in URLs (length, symbols, domain structure, and so on).

Dataset Splitting

The dataset may be subdivided into three subsets that are not overlapping to make the evaluation process fair and unbiased:

Training set (~70%): Training set is used to learn models.

Validation set (~20%): This is used in tuning the hyperparameters and keeping track of the performance.

Test set (~10%): Applied to the final test of unknown data.

This division helps to make sure that the model is trained on enough data and at the same time, the benchmark is reliable in terms of the generalization performance.

CHAPTER-4

PROPOSED APPROACH , MODULES DESCRIPTION, AND UML DIAGRAMS

4.1 Proposed Approach

The suggested system is aimed at creating an automated system with the capability to process the documents and links that are shared on WhatsApp in a smart and safe manner. Everyday communication involves users sending valuable links, PDFs, and other files via messaging services, yet they are lost or not easily arranged and accessed in the future. This system will help address that issue by automatically processing the shared content and extracting valuable information and storing it in a structured and searchable form.

This starts when a user communicates with the system through WhatsApp. This message can include various kinds of input that can include a web link, a PDF file, an image or a document in such formats as DOCX or TXT. After receiving the content, the system takes into account the input by first acknowledging it and thereafter begins processing it in stages.The initial process in the workflow is to determine the kind of content that has been posted. This is significant since various kinds of inputs will have varying processing methods. An example is that a web link must be read and interpreted whereas a document file must be read and interpreted. The system relies on the simple detection techniques like file extensions, metadata and pattern matching to categorize the input accordingly.

Once the type of content has been identified, the system converts all the inputs into a standard format to facilitate easier processing of the same. In this case, PDF is used as the common format. In case the user uploads an image or a document file, it would be converted to a PDF. In the case of images or scanned documents, Optical Character Recognition (OCR) is used to extract text out of the image and then converted. This process will make sure that any content no matter the form it is in can be treated in a similar manner.

After standardization of the format, the system conducts a security check to ascertain that the content is safe. This is a significant measure since users can post malicious links or infected files unknowingly. In the case of URLs, the system verifies whether the link is suspect based on the structure of the link, and comparing it with the data of malicious links. In the case of documents, simple forms of scanning are employed to identify potentially malicious content. In case of any threat being detected, the system may block the content or warn the user.Once it has been established that the content is safe, the system will then check whether it has access to the

content or not. Certain links might be password-protected or certain files are to be logged in with credentials. When this happens, the system detects these restrictions and treats them as such; by either ignoring the content or informing the user of the restriction.

Content extraction is the next step. In the case of web links, the system selects the relevant sections of a webpage like the title, headings and main body text and leaves out other irrelevant webpage content, such as advertisements or menus, etc. In the case of PDF files, the system parses the files with the help of parsing tools and in case it is scanned, OCR is employed to read the document. The objective of this action is to get the key information in a clean and usable format.

After extraction of the text, the system uses basic natural language processing to get a better understanding of the text. These involve tokenization (subdivision of the text), the elimination of common words which do not convey much information (stop words), and the reduction of words to their root forms. These are the steps that assist in the preparation of the text to be analyzed further.

Following the preprocessing the system will extract significant keywords within the content. Such keywords are the primary concepts of the document or link. Besides the use of keywords, the system can also identify certain entities like names of individuals, organizations or destinations. This aids in developing a further insight into the content.On the basis of this analysis, a set of tags is created that characterizes the general subject of the content. These labels are used as tags which enable the information to be organised and searched later. To illustrate, a document on the topic of artificial intelligence could be marked with such keywords as machine learning, data science, or AI. Such tags are produced automatically with either simple algorithms or pre-trained models.

The processed content, and its extracted information and tags are finally stored in a database. The details that the system keeps include the title, summary, tags and the original source. This organized storage enables users to readily access the information at any time.

In case the user needs to locate a document or link that they shared before, it is possible to do a search with the help of keywords or topics. The stored tags and content are used to provide the most relevant results using the system. This renders retrieval process quick and efficient even in cases where a huge amount of data is stored.

In general, the suggested solution is set to be easy, effective and convenient to use. The automation used together with a little intelligence allows the system to save the effort needed to maintain shared content and allows the user to easily access valuable information whenever he or she needs it.

4.2 Modules

The system will be developed as a modular system whereby each module will have a particular role in the entire workflow. This enhances better organization of the system, easier debugging and enables each component to be developed independently. The modules are sequential with the output of one module being the input of the other module.

4.2.1 Input Module

The System has the Input Module as its entry point. It takes care of the user data in the mobile application interface. Input can be made by the users as web links or PDF documents.

This module will only accept valid and correctly formatted input and consequently send it to the back-end system.

This module has main functions that include:

- Taking user input (link or document)
- Checking format and file type of links.

Checking file size limits

Preventing invalid or empty submissions.

The input is also converted to a structured format like JSON by the module. Each input is assigned with a special ID and a time stamp which makes it traceable. This is structured data which is transmitted to the backend through API calls.

4.2.2 Module of validation and Safety.

The Validation and Safety Module will make sure that only safe and trusted content is processed by the system. It is very important in safeguarding the system against malicious inputs.

This module examines the links and documents and then proceeds with processing.

Some of the main activities carried out comprise:

- Check of URL and accessibility.
- Domain checking and regularity checking.

Features: Linking suspicious or unsafe links may be detected.

- Format and integrity of files validation.
- The process of detecting a blocked or closed content.

In case any input has been detected to be unsafe, the system blocks the input and responds appropriately to the user. Otherwise, the content is forwarded to next stage.

This module not only makes systems more secure, but it also makes it safe to continue the

execution of other processes.

4.2.3 Content Extraction Module

Content Extraction Module is charged with the responsibility of extracting valuable information out of the validated input. It transforms raw data into readable meaningful text which can be analyzed.

In the case of web links, the system identifies significant parts of the web links like titles, headings and main content. In the case of PDF documents, the parsing techniques are used to extract the text or OCR in the case of scanned files.

Main functions include:

- Extracting text from web pages and documents
- Elimination of redundant items such as advertisements or programs.

Clean text: Removing special characters.

- Formatting mined data to be used further.

This module will make sure the data transmitted to the next process is only relevant and clean so that the system will be more accurate.

4.2.4 NLP Processing Module

The NLP Processing Module normalizes the text extracted to be analyzed by using natural language processing. It assists the system to know the content in a systematic way.

The module does a number of preprocessing to enhance the quality of data.

Key steps involved:

- Tokenization (text breaking up into words)
- removal of stop-words (elimination of common words)

Stemming or lemmatization (reducing words to root form)

- Important terms frequency analysis.

The processes minimize noise in the data and emphasize crucial information. The result is a polished copy of the text, to be used in the production of tags.

4.2.5 Tag Generation Module

The Tag Generation Module is an essential part of the system that creates meaningful tags that express the content.

This module involves NLP and machine learning to extract important topics and concepts out of the text that is being processed.

Main operations include:

Part-of-speech tagging: This technique is used to identify keywords.

- Interpretation of semantic meaning of the material.
- Ranking keywords on the basis of relevance.
- Choosing the most pertinent tags.

The resulting tags can be used to sort the data, and enhance search capabilities. This module allows the system to be intelligent by allowing automatic categorization.

4.2.6 Database Module

The Database Module will be tasked with storing all the processed data in a format. MongoDB is used due to its flexibility and scalability.

This module contains different kinds of information which is needed by the system.

Data stored includes:

- User information
- Documents and links
- Extracted text content
- Created metadata and tags.
- Processing status

The database is constructed in such a manner that it is fast in responding to queries and indexing. This is to guarantee speed in retrieving data as users make searches.

4.2.7 Retrieval Module

The Retrieval Module enables the users to find and retrieve stored content in an efficient manner. It is also important in enhancing the user experience.

As a user makes a query, the module forms the query and then gets the appropriate data in the database.

This module has functions that include:

- Taking search queries of the users.
- Comparison of queries with data in stores.
- Search with keywords/ tags.
- Sorting results in relevancy.

The module will provide the users with quality and relevant results within a short time.

4.2.8 User Interface Module

User Interface Module deals with communication between user and the system. It is created with React Native to deliver an excellent mobile experience.

This module consists of different screens and features, which simplify the system.

Main features include:

- Registration of the users and their log-in.
- Monitor showing data about the system.
- Document upload functionality
- Search and access interface.
- Navigation menu to easily access.

The interface interacts with the backend via APIs, and reflects results dynamically. It is developed to be easy, reactive and user friendly.

4.2.9 Module Interaction Flow

The system modules are set to operate sequentially and interdependently. All the modules rely on the results of the last module, creating a pipeline which converts raw input into meaningful and structured data.It starts with the input of the user who inputs either a link or a document via the Input Module. This information is then forwarded to the Validation and Safety Module which verifies content as to whether it is safe and appropriate to process.

The data that is extracted is then subject to the NLP Module to eliminate irrelevant information ready to be analyzed. Once the preprocessing is done, the Tag Generation Module will extract the important topics and create meaningful tags. The processed data as well as tags is then stored in the Database Module.

Lastly, the Retrieval Module enables users to view the content that has been stored and the User Interface Module presents the results in a structured format.

4.2.10 How the Data Flows Among the Modules.

The system has a systematic flow of data and thus every module is supplied with clean and pertinent input data.

The flow may be characterized as:

- User makes input by use of mobile application.

Input: Input is checked and formatted.

- Information is verified as being safe and reliable.
- Extracted and cleaned content.

Text: NLP is used to process text.

Tags: Tags are created according to content.

- Information is saved on database.
- Outcomes are presented to user.

The data is refined by each stage, to enhance accuracy and usability. The system makes sure that only meaningful data that has been processed is stored and retrieved.

4.2.11 Benefits of a Modular Design.

Modular design has a number of advantages to the system. The modules are independent, thereby allowing the modules to be easily developed and maintained.

It has some of the following benefits:

- Better organization and visibility of the system.
- More convenient debugging and testing of modules.
- Ability to update or change modules.

Better ability to scale to enhancements in the future.

- Fewer complexities in system design.

This way developers can work on a single aspect of the system at a time, and the development process will be more effective.

4.2.12 Module Dependency Explanation

The modules of the system are interdependent on the correct operation of the previous modules. Failure of one module may have an impact on the whole system.

For example:

- In event of failure of the Input Module, no data will be fed in the system.
- In case of the malfunction of the Safety Module, malicious data might be processed.
- In case of failure of the Extraction Module, there will be no helpful content.
- Incase of failure of NLP Module, generation of tags will be inaccurate.

Thus, it is necessary to coordinate and handle the errors in inter-module coordination. The system is made to cope with these problems by validation of outputs at every stage.

During the implementation of a module, it is required to have error handling.

To maintain smooth operation of the system, error handling is an aspect of the system that is important. All modules have error detecting and error handling mechanisms.

The typical errors situations are:

- Invalid input format

- Missing or invalid links.
- Unsupported file types
- Inability to extract content.
- Incorrect tag generation

The system copes with such errors by:

- Showing the relevant error messages.
- Skipping invalid inputs
- Unless use of logging errors to debug.
- Preventing system crashes

This will make sure that the system will not go out of proportion even in cases where there are unanticipated problems.

4.2.14 Scalability of the modules and Future improvement.

The system is scaled and expanded easily because of the modular nature of the system. The additions of new features do not require any changes to the whole system.

Possible enhancements include:

- Supporting additional types of files (images, videos).
- Enhancing the creation of tags with state-of-the-art AI models.
- Integrating real-time notifications
- Semantic matching of search.
- Adding cloud-based scalability

The system is planned in such a manner that these enhancements can be done with few modifications to the current modules.

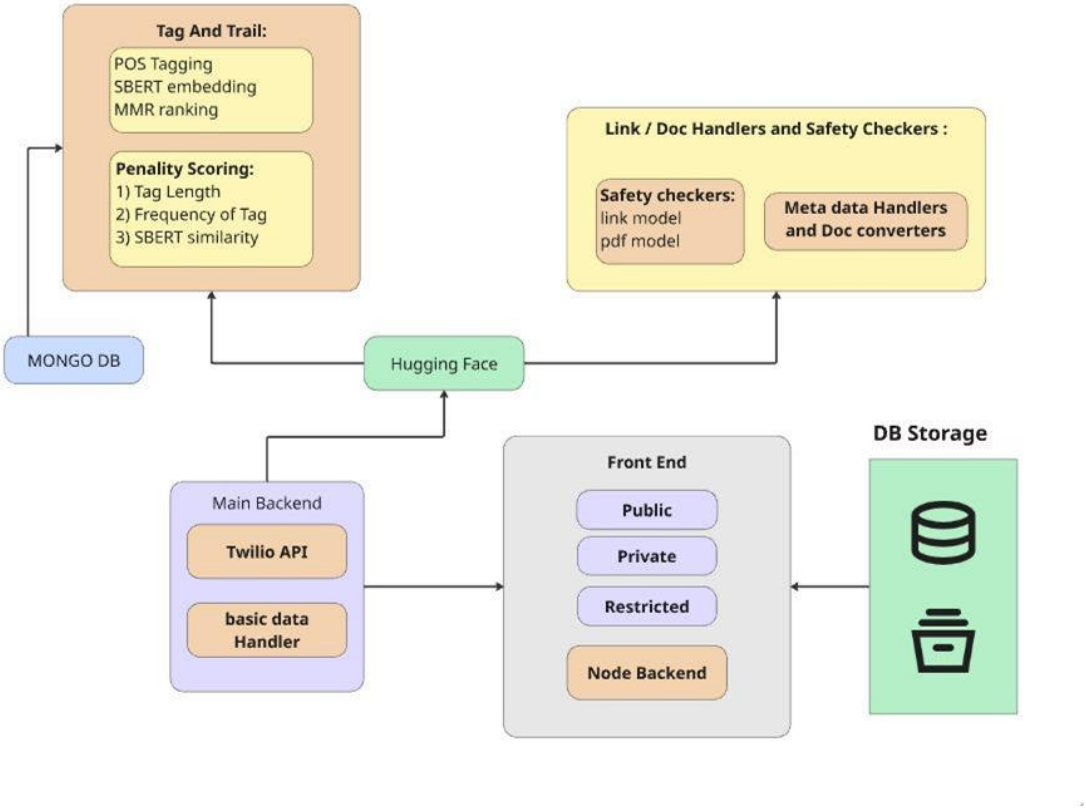
4.2.15 Workflow (Real Life) Example.

In order to comprehend the working of the system, we can take the following example:

One of the users provides a web address via the mobile app. The input Module gets the connection and transmits it to the backend. Validation Module: This checks on whether the link is safe. Upon approval, the Content Extraction Module gets the text of the webpage.This NLP Module works on the text and eliminates the redundancy of words. The Tag Generation Module then finds significant words and creates tags like: technology, AI or education. This information is stored in database.Subsequently, the user can query the Retrieval Module with the query AI and the results are retrieved and presented to the User Interface.This illustration is to show the entire functionality of the system.

4.3 UML Diagrams

System Architecture:



4.3.1 System Architecture

i) Overview of the Architecture

The suggested system is developed as a scalable and modular architecture that optimally processes the documents received via WhatsApp and their connections. The primary aim of the architecture is to provide a fluent movement of information between user input and end storage keeping security, consistency and smart processing. In order to accomplish this functionality, the system combines several components like APIs, backend services, machine learning models, and database systems.At a high level, the architecture is a pipeline type with each component executing a particular task and forwards the processed data to the next stage. The system works with the user via WhatsApp and the message is received with an API service. This input may be in the form of a link, document or file which is then processed sequentially. The system makes sure that any form of inputs is processed in the appropriate manner and converted to a standard system that will be processed uniformly.

The architecture is made to be flexible, such that in future it is possible to add new features or modules without impacting the existing system. It is also user-friendly to all user types and access levels, including the public, private and restricted access levels, and thus it is limited to

a variety of use.

ii) Workflow and core components.

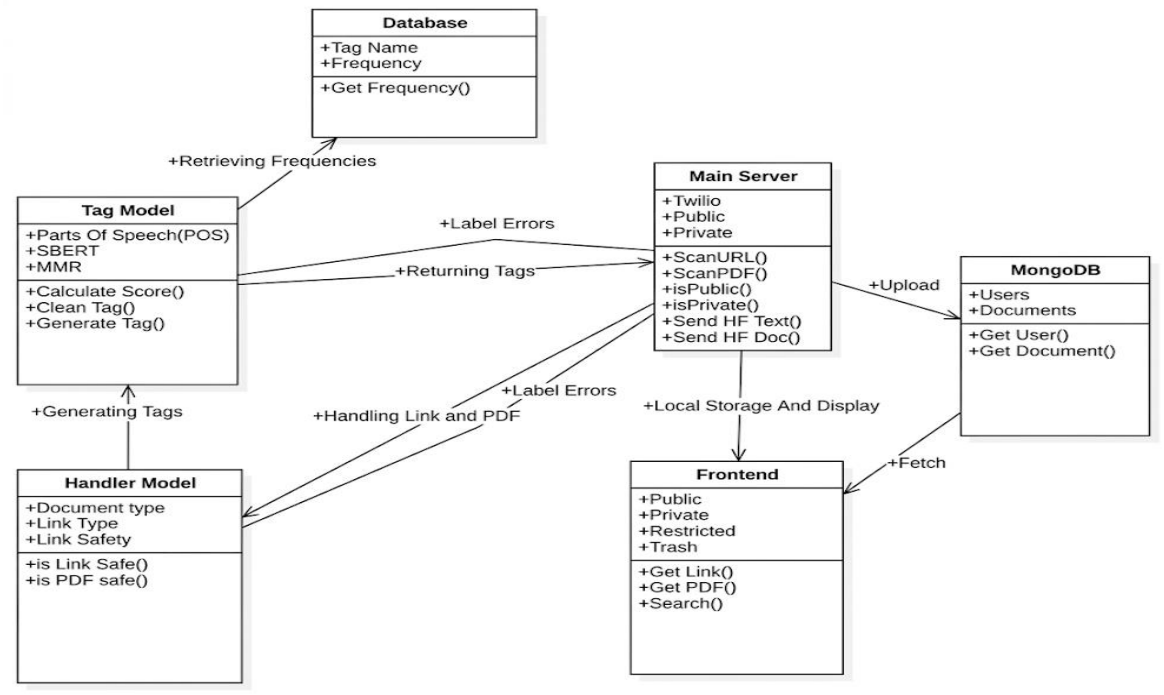
The system is a network of a number of connected elements which collaborate to process the input data. The initial critical element is the backend system which serves as the central point of control. It takes the input of WhatsApp via the Twilio API and executes first functions like the validation of the input and determining its type. Depending on the type of input, the data is sent to the relevant processing modules by the backend.The second element is the connection and document management as well as safety checking systems. This component of the system makes sure that the content undergoing processing is safe and usable. Cyber checks are done on links with models to identify malicious or phishing content, and documents are scanned to identify possible threats. Meanwhile, this module pulls out metadata and transforms unsupported file formats to PDF, making sure that there is consistency across the system.Once validated, the content is sent to the machine learning and NLP layer where further analysis is conducted. This layer makes use of models to derive meaning of the text as opposed to reading it.Lastly, all the processed data, with or without the original content, extracted text, metadata and generated tags are stored in the database. The data stored in MongoDB is structured and semi-structured and is efficiently stored. The stored data can be then accessed by the frontend which provides various access levels depending on the user permissions.

iii) Information flow and major characteristics.

The general flow of data in the system starts when a user messages using WhatsApp. The backend receives this message via the API and the message is validated and categorized. The data is then subjected to the safety checking and conversion modules to check whether the data is secure and in the appropriate format. Then, machine learning models are used to analyze the content and extract meaningful information and generate tags. Lastly, the data that has been processed is stored in the database and can be retrieved.Security is also of high concern with in-built systems to recognize malicious links and documents prior to their processing. Moreover, machine learning and NLP techniques transform the system into a more intelligent one which can comprehend the content on a higher level and create correct tags.

In general, the system architecture offers a properly structured and effective system of document and link processing. It has the advantage of making sure that data is handled in a safe way, intelligent analysis and storage in an organized form hence it is easy to retrieve data and even more effective to the users.

Class Diagram:



4.3.2 Class Diagram

The system consists of multiple components that will be outlined below:

The system is developed with various interrelating parts with every part tasked with a particular task in the general process. This modular design aids in the simplification of the design and enables the system to be easier to handle, test and expand in the future. The architecture primarily comprises of the Main Server, Tag Model, Handler Model, Database, MongoDB storage and Frontend interface.The Main Server is the central point of the system; this is the control unit of the system and is the one that coordinates all the other components. It communicates with other services, such as Twilio to get WhatsApp messages and controls the data flow between processing modules and storage systems.The processing and analysis of the content is done by the Tag Model and Handler Model. Whereas the Handler Model is concerned with validation and processing of input like links and documents, the Tag Model is implemented to create meaningful tags out of the processed content. Assisting these modules is a Database which holds tag-related information like tag names and frequency which helps enhance quality of generated tags with time.This system is also equipped with MongoDB that is the primary storage system of both users and documents and a Frontend interface that enables users to

access, search and view stored content in various categories according to their accessibility and security levels like public, private or restricted.

ii) Core Modules Working.

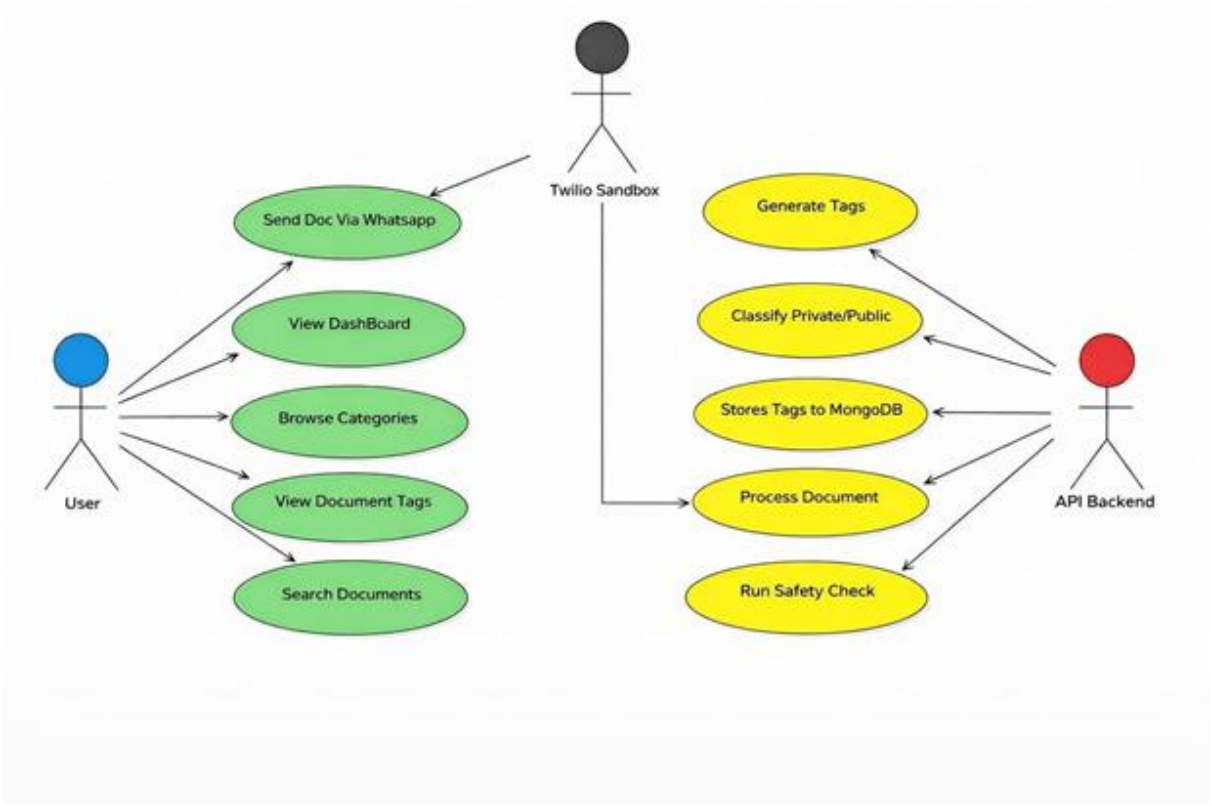
The workflow is initiated when the user initiates a message or file via WhatsApp. This input is received by the Main Server using the Twilio API. The server then carries out preliminary searches like URL or PDF scanning and identification of the content as being public or private. It also garner out fundamental information like text in links or documents.After the input is given, it will be sent to the Handler Model which will analyze the kind of input it is and make it safe. This module verifies that the link is not a malicious one and that the PDF is not malicious to be processed. It also determines the type of document and type of links. In case of any problems or errors identified during this phase it is reported back to the Main Server to be dealt with.Once the content is confirmed; it is sent to Tag Model, which is in charge of creating meaningful tags. The content is analyzed using Parts of Speech tagging, SBERT embeddings and Maximal Marginal Relevance in this model. It computes the potential tags scores and weeds out irrelevant ones. The Tag Model can also access the Database to obtain the tag frequency data to enhance tag selection. According to this analysis, it comes up with a collection of tags that most accurately describes the content.

iii) Data Flow and Interaction with the system.

After the processing process is complete, the Main Server uploads the information to MongoDB where user data and documents are stored. This contains information like extracted text, created tags and type of document. MongoDB offers an effective means of storing and retrieving mass structured and semi-structured data.The Frontend communicates with MongoDB to access data kept and present it to the user. It also offers various perspectives including public, private, restricted and a trash area where deleted content can be found. Keywords or tags allow users to search documents or links, and it is easier to find the previously stored data.

The information flow within the system is rather straightforward: the Main Server is the place where the input is received and then the Handler Model is used to validate it and the Tag Model is used to create tags and then is stored in MongoDB. This information is then retrieved by the Frontend, and presented as necessary.

Use Case Diagram:



4.3.3 Use Case Diagram

i) Overview of User Interaction

The use case diagram, illustrates the interactions between the various actors and the system and how the various functionalities are implemented. This system has three key participants: the user, the twilio sandbox, and the API backend. The actors have a particular role to play in the smooth running of the system between the input and the output.The main player in the system interaction is the user who communicates with the system by using WhatsApp, or a related interface. The user is able to do various tasks including document sending, dashboard viewing, browsing, document tag checking and searching. These operations portray the sole objective of the system, which is to assist users to handle and access documents easily.

The Twilio sandbox is an interface that facilitates communication between the system and the user. It accepts user- messages and sends them to the backend to be processed. This enables the system to operate without having to be accompanied with a discrete application interface, which is more convenient to users.

ii) Processes in the System.

When a user sends a document via WhatsApp, system initiates a series of actions. The message is passed to the Twilio sandbox and it is relayed to the backend. The backend then begins to process the document and extracts its content, and analyzes its structure.A safety check is one of the initial activities in the process. This makes sure that the document/link offered by the user is safe and free of any virus. Once it is verified that it is safe, the system then goes ahead to process the document extracting the relevant text and information.

After extracting the content, the system categorizes the content in terms of either public or private. Such classification aids in regulating access and organization of the data in an efficient manner. The system then produces tags which explain the key topics of the document based on the content. These labels are significant as it is much easier to search and filter.

Upon tagging the documents, the system records them and the document information in MongoDB. This will make sure that information processed is stored in an organized manner to be utilized later on. This is a fully automated system with the backend taking care of this entire workflow and not having to be handled manually.

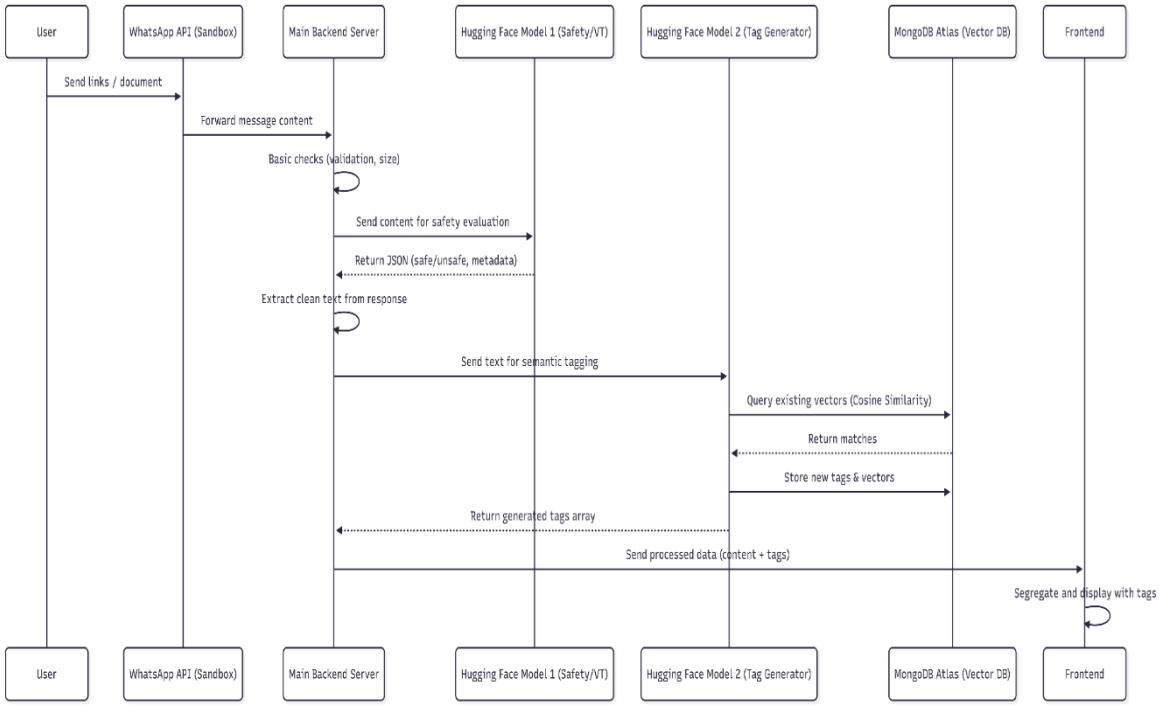
iii) User Access and output.

Once the process of processing the document and storing it is done, the user can then communicate with the system to retrieve information. The user is able to view a dashboard where a summary of documents which have been stored can be seen and the type of documents stored. They are able to navigate in various sections to get the related material or see the tags related to each document.The search system enables one to fasten search of particular documents by keywords or tags. This ensures that the process of retrieving the documents is quick and easy even when numerous documents had been stored in the system.

The state of the interaction between the system and the user is to be simple and effective in general. All one has to do is to send a document or a link and the system will process, analyze and organize the data. This saves time of manual work and enhances productivity.

In general, this architecture offers a properly organized system of managing document and link processing. It makes sure that the content is safely processed, intelligently analyzed and stored in an efficient way and also gives users an easy access and control of their data.

Sequence Diagram:



4.3.4 Sequence Diagram

The flow of interaction is represented above.<|human|>i) Interaction Flow Overview.

The sequence diagram is used to show the interaction of various system components in a given step by step when a user submits a document or link. It also has a clear indication of the flow of data through different layers, beginning with the user and concluding with the frontend display. The key stakeholders in this process are the user, WhatsApp API (sandbox), main backend server, two machine learning models, MongoDB database, and the frontend interface.It starts when the user transmits a document or a link on WhatsApp. The WhatsApp API is the first point of entry into the system which receives this input. The API then directs the content of the message to the main backend server where it is processed further. The components in the series carry out a particular task and transmit the outcome to the other component and this forms a chain of operations.This formal communication will provide the system with an efficient way of handling user input and also the system will coordinate well among the various modules.

ii) Data Processing, Step-by-Step.

After the content is received by the backend server, it undertakes some basic validation processes like checking the size of the file, format and simple correctness of the file. These

checks make sure that the system does not process invalid and unmanageable inputs. After validation, the content is sent to the first machine learning model, which is responsible for safety evaluation.The safety model processes through the content and responds by giving out a reply that it is safe or unsafe. It can also supply extra metadata, in conjunction with this. The backend then removes irrelevant and dirty text in this response ready to be analyzed further.The processed text is then passed to the second machine learning model which is concerned with semantic tagging. This model creates good tags out of the content. It further uses similarity measures like cosine similarity to compare the generated embeddings to the known vectors in the database thus enhancing accuracy. The system finds matching tags using this comparison and narrows down the tags.

Once the final set of tags has been created, the model inserts the tags and the vectors representing them in MongoDB. This assists in enhancing the search and recommendations in the future. A processing stage is then done by sending the generated tags back to the backend server.

iii) Storage and Final Product.

Upon getting the processed data in the tagging model, the backend uses the obtained content and tags to combine them. Information and this is sent to the front end interface to be displayed.Simultaneously, the data is stored in MongoDB Atlas that is a vector database. The storage enables the use of both textual data and embeddings to be handled efficiently enabling the semantic searches to be carried out in the future. The database makes sure that all documents processed are stored with the relevant tags and metadata.At the front end, the system views the data and puts it into a clear format to the user. The documents are presented with their tags and a user can easily comprehend and classify the information. The frontend can also isolate documents on the basis of various criteria which make navigation easier.

Generally, the sequence diagram shows the system follows a clear sequence in processing the user input. All the components help to manipulate raw input into valuable and structured information. Such step wise interaction will provide accuracy, efficiency and unhindered user experience within the system.

CHAPTER-5

5.1 IMPLEMENTATION

Project Overview:

TagAndTrail is an intelligent document and link management system designed to make the operations users perform on shared digital material easier, through the use of mobile technology. In the modern world, users often get access to significant links and documents via such applications as WhatsApp, and they are easily lost or hard to arrange. The proposed system will eliminate this problem by offering an automated solution that will process, analyse and classify content in a systematic way.The system will execute several functions that include the identification of the type of input, verification of the content safety, extraction of meaningful information and semantic tagging. These tags assist in sorting the material and enable retrieval to be quicker and more effective. The app also offers an interactive and clean mobile interface that enables users to easily manage the data they have stored.

The entire system is developed based on a mix of technologies such as React Native in the frontend, Node.js in the backend to deal with API, Python in AI processing, and MongoDB to store data. With the addition of machine learning models, the system will be more intelligent, as it will be able to comprehend the content on a deeper level.

Requirement Gathering and Collection of Resources:

The process of the development started with comprehending the central issue and determining the necessities that should be fulfilled to create a successful solution. The overall aim was to develop a system that has the ability to automatically process documents and links without necessarily organizing it manually.

At the research stage, several issues were examined, such as the document processing techniques, natural language processing methods and link safety detection mechanisms. Analysis of existing systems was done to know the shortcomings of the systems and how they could be improved.Sample datasets (in the form of web links and PDF documents) were gathered to test the system. These datasets contained various forms of content like educational links, project documents and general web pages. The gathered data was made available to assess the effectiveness with which the system would extract the information and produce pertinent tags.

Besides datasets, other resources like APIs, libraries, and development tools were also found. These were machine learning text analysis models, APIs to communication, frontend and

backend development frameworks.

Preliminary Development and System Design:

A proper system design was developed in advance to establish the interaction between various components before it began the actual implementation. The system was subdivided into several modules to make it clear and maintainable.

The design included:

- A user interface module.
- A request-handling and routing backend.
- A content analysis AI processing module.
- A database component to store processed information.

The architecture adopted was a modular one, in order to have each component developed independently. This further simplifies the process of updating or replacing individual components in future without impacting the whole system.

The data flow and the interaction between the components were visualized with the help of flow diagrams and architecture diagrams. This aided in the detection of the possible problems at an early stage of development.

Frontend (React Native):

React Native was used to create the frontend of the application to enhance a user-friendly and responsive interface. React Native was selected as it is a cross-platform language that uses components-based architecture.

The frontend has a number of key features:

- System of user authentication with registering and logging in.
- Dashboard that shows an overview of the data stored.
- Adding new documents or links, option.
- List view of classified data.
- Quick access search.

All the features were applied individually and it made the code more structured and reusable. To illustrate this, the document card element is used to show links and PDF documents and their tags.

The user interface design was given special consideration, so that the application is easy to use. To add to the overall experience, a regular color scheme, adequate spacing, and fluid animations were applied.

The front-end also uses API calls to the backend to retrieve and show the data dynamically. This makes sure that there is continually updated information before the user.

Backend Development (Node.js):

The system back-end was done with Node.js and Express.js. It serves as the main controller which controls communication between the AI module, database and frontend.

The backend will be in charge of:

- Getting requests sent by the frontend.
- Validating input data
- Makes requests to appropriate modules.

A python AI service which enables communication with the service.

- Response the answers to the frontend.

RESTful APIs have been developed to accommodate different activities such as the insertion of documents, data retrieval and even the user information. Such APIs make sure that communication between various components of the system is smooth.

Another component of the backend is error management where the system is not brought to a stop when all other unexpected issues occur.

AI Processing Module (Python):

The AI processing module is one of the most important features of the system. It is coded in Python and does the content analysis and tagging.

There are various processes that are conducted in this module:

- Estimating the security of a link or document.

Reading input text out of the input.

Processing of text by NLP methods.

- Generating meaningful tags

The context of the content is comprehended using machine learning models. Keywords are not extracted in the system but the meaning of the text is processed and tags created.

This module is the output of a product in the form of a JSON containing information on the safety status, extracted text and generated tags. It is then stored and displayed in the form of data by the backend.

Database Implementation (MongoDB):

The database was stored in MongoDB Atlas in which all the application data is stored. It is a

NoSQL database which enables the flexibility of a structured and semi-structured information storage.

The database stores:

- Information of users such as login details.

Checklists and links posted by users.

- Developed metadata and tags.

Processing results and status.

The schemas were planned in a way that guarantees consistency in storing the data. This makes accessing and working with data easier.

Another benefit of MongoDB is that it is also able to perform rapid querying enhancing the speed of the system when searching documents.

Integration and System Workflow:

The system is a combination of all the components to form a complete workflow. It starts when one of the users is saving a link or document through the mobile application.

Input is sent to the backend which does preliminary validation. The information is then forwarded to the AI module whereby safety tests and tag creation are carried out. The results are stored in the database immediately the processing is completed.

Finally, data is processed and the frontend receives the processed data and makes it available to the user in a structured format. The entire process is automated and reduces the human input in the process and boosts efficiency.

Debugging and Optimization:

When developing, the following challenges were experienced. These were related to API communication problems, data synchronization, and wrong AI model results.

To come up with a solution to these issues, a number of debugging techniques were used, such as:

- Logging system outputs
- Testing individual modules
- Verifying API responses

Fixing UI rendering problems.

Performance was also enhanced using optimization techniques. This included reducing the amount of unnecessary API calls that were being made, improved management of data and optimization of data search in the database.

Last Integration and Testing:

After the completion of all the parts the system was assembled and well tested. The systems were verified in terms of functionality through the assistance of various kinds of inputs.

Testing included:

Checking of the processing of documents is correct.

Checking Accuracy of tag generation.

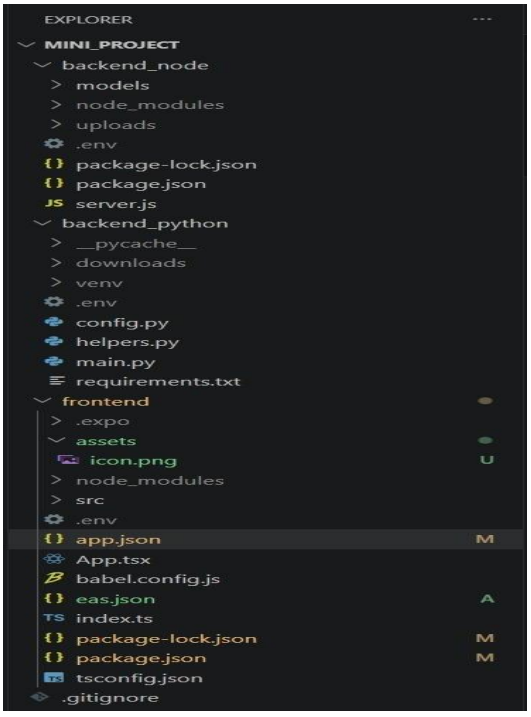
- Proper storage in the database.
- Testing responsiveness of the user interface.

The results showed that the system is effective and provides accurate results. The application also underwent testing on other devices to make sure that it was compatible.

Implementation Summary:

It is demonstrated by the implementation of TagAndTrail that different technologies can be united to create a smart system. The system is made functional and scalable by using frontend, backend and AI.

The project works well in regards to automation of the document processing and enhancement of the data organization. It offers an effective way of organizing digital material in an effective and efficient way.

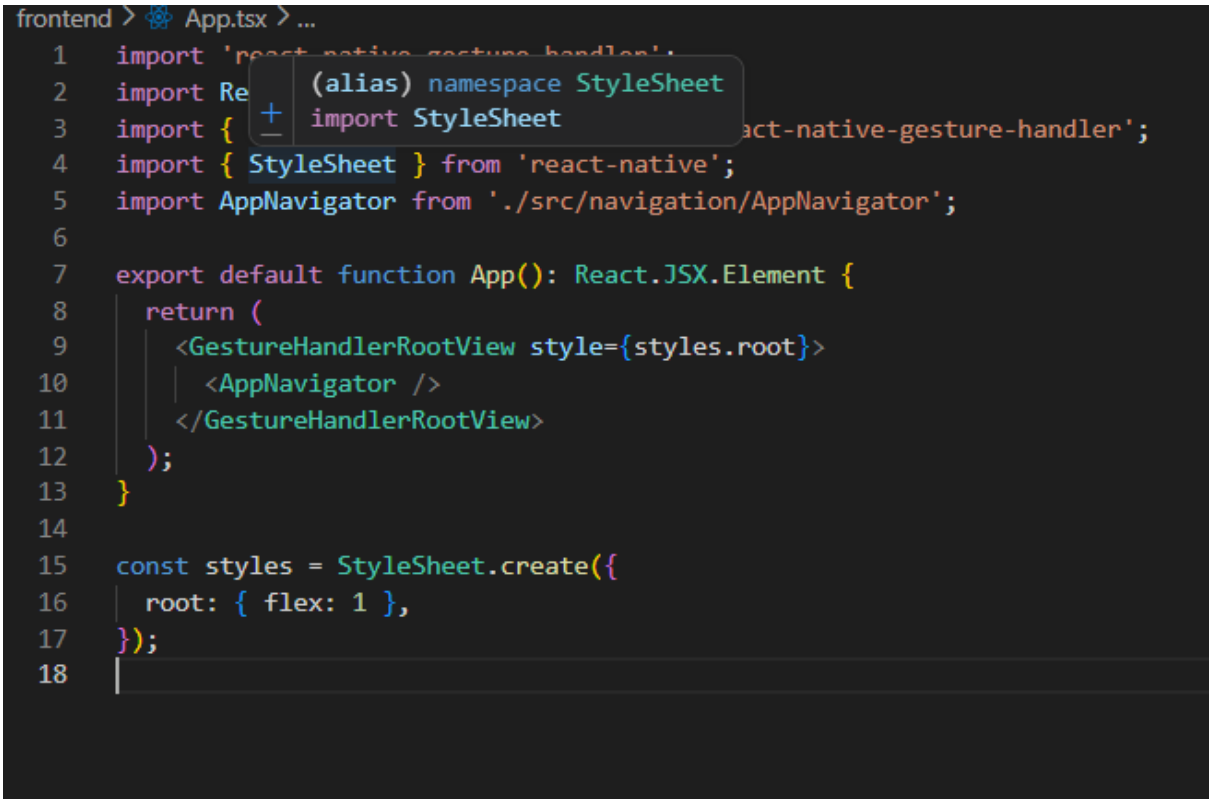


5.1.1 Project Folder Structure

Following figures illustrates core implementation of the project containing frontend, backend , database and Api connections

Frontend:

i) App.tsx



5.1.2 App.tsx

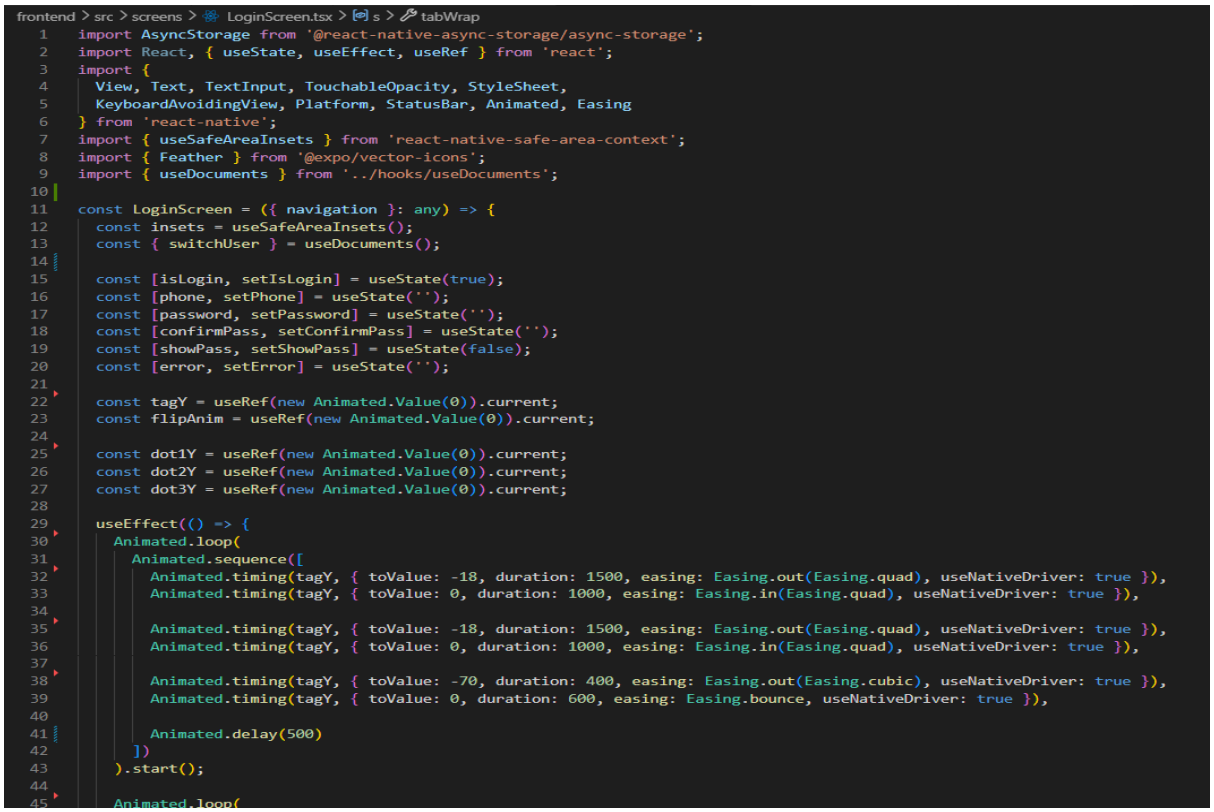
App.tsx is the main entry of the React Native application. It starts the application and establishes the navigation system.

The app is encased by GestureHandlerRouter that allows gesture interactions like swiping and tapping throughout the app. AppNavigator component takes care of navigation across various screens such as screen such as login, dashboard and document view.

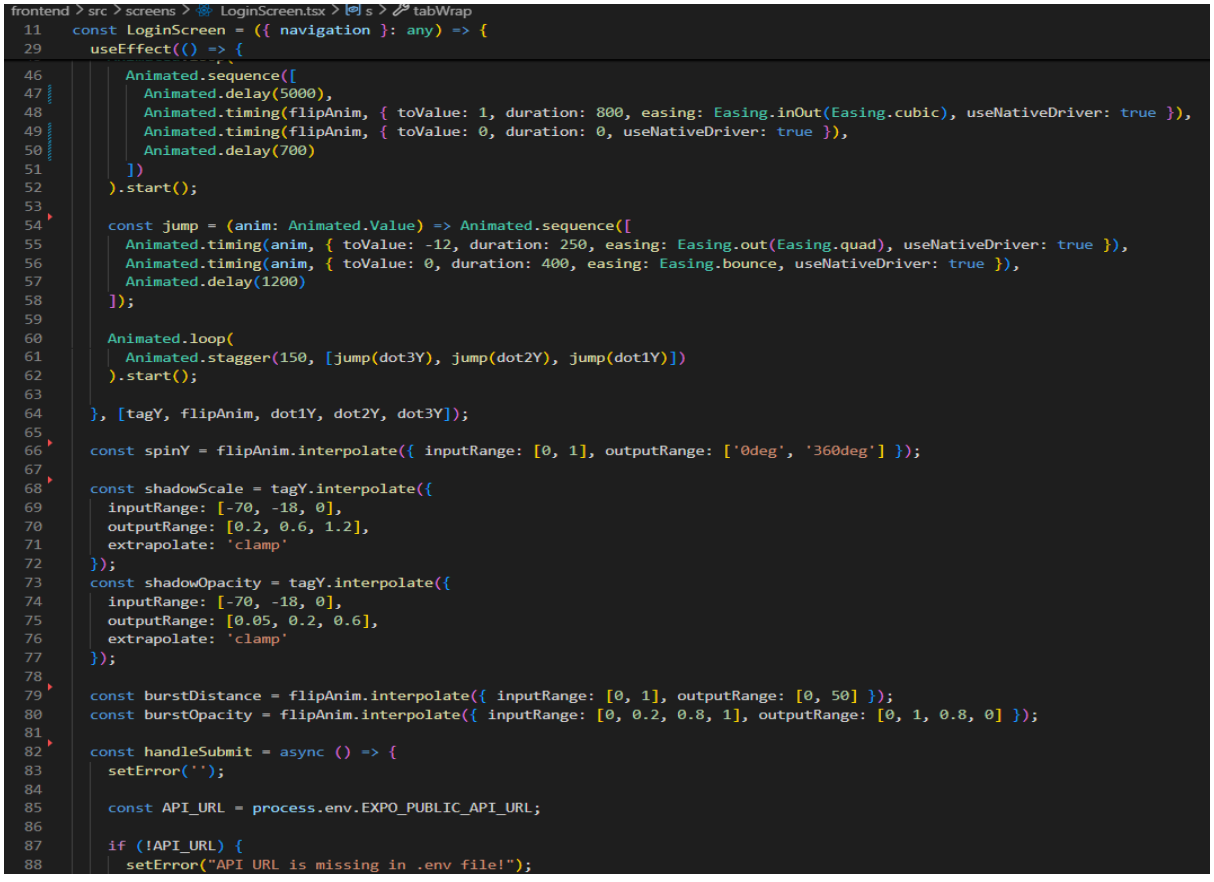
A simple style is made with the help of StyleSheet, with the root container being flex setted to occupy the entire screen with flex:1.

All in all, this file provides the application with proper initializing, navigation and gesture control.

ii) LoginScreen.tsx

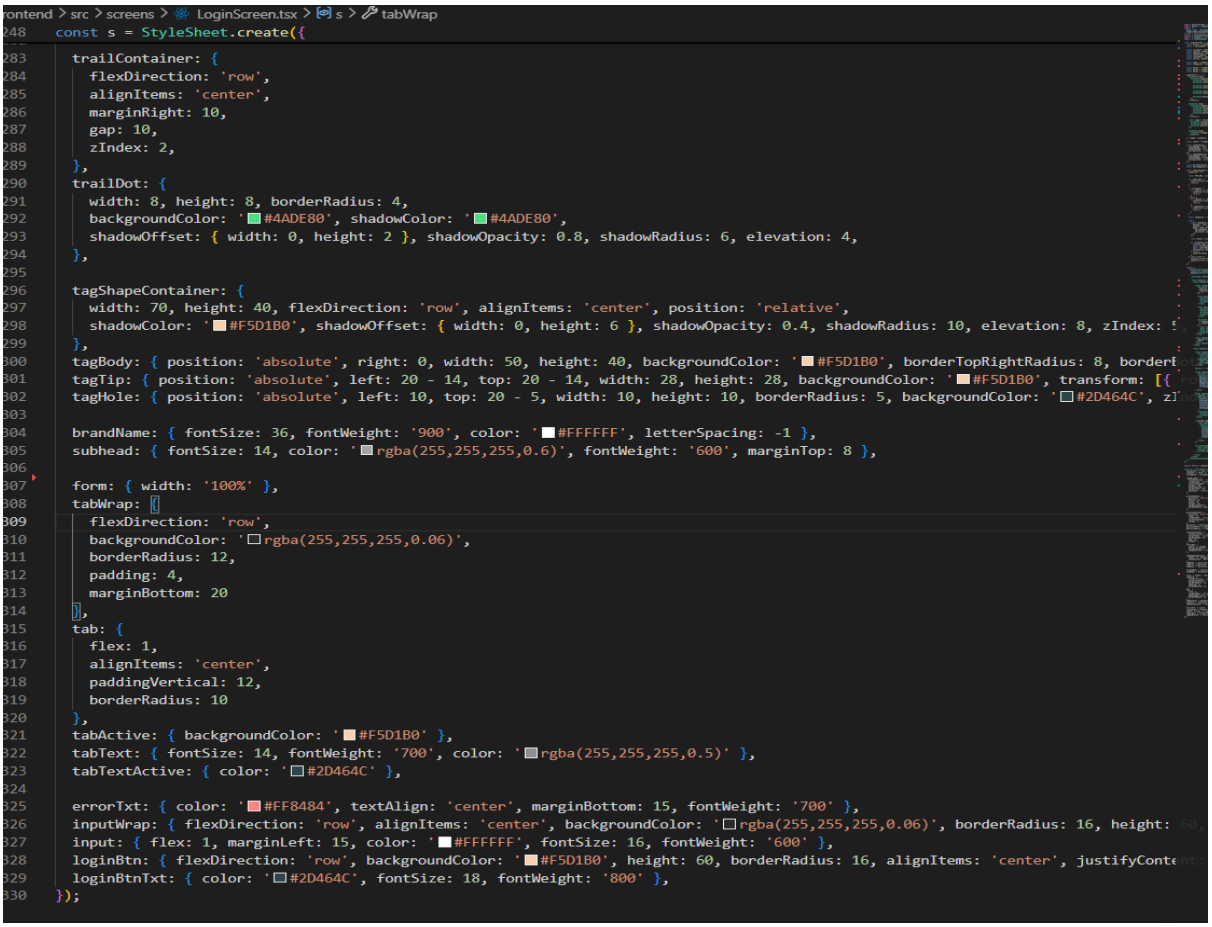


```
frontend > src > screens > LoginScreen.tsx > s > tabWrap
import AsyncStorage from '@react-native-async-storage/async-storage';
import React, { useState, useEffect, useRef } from 'react';
import {
  View, Text, TextInput, TouchableOpacity, StyleSheet,
  KeyboardAvoidingView, Platform, StatusBar, Animated, Easing
} from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { Feather } from '@expo/vector-icons';
import { useDocuments } from '../hooks/useDocuments';

const LoginScreen = ({ navigation }: any) => {
  const insets = useSafeAreaInsets();
  const { switchUser } = useDocuments();

  const [isLogin, setIsLogin] = useState(true);
  const [phone, setPhone] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPass, setConfirmPass] = useState('');
  const [showPass, setShowPass] = useState(false);
  const [error, setError] = useState('');

  const tagY = useRef(new Animated.Value(0)).current;
  const flipAnim = useRef(new Animated.Value(0)).current;

  const dot1Y = useRef(new Animated.Value(0)).current;
  const dot2Y = useRef(new Animated.Value(0)).current;
  const dot3Y = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.loop(
      Animated.sequence([
        Animated.timing(tagY, { toValue: -18, duration: 1500, easing: Easing.out(Easing.quad), useNativeDriver: true }),
        Animated.timing(tagY, { toValue: 0, duration: 1000, easing: Easing.in(Easing.quad), useNativeDriver: true }),

        Animated.timing(tagY, { toValue: -18, duration: 1500, easing: Easing.out(Easing.quad), useNativeDriver: true }),
        Animated.timing(tagY, { toValue: 0, duration: 1000, easing: Easing.in(Easing.quad), useNativeDriver: true }),

        Animated.timing(tagY, { toValue: -70, duration: 400, easing: Easing.out(Easing.cubic), useNativeDriver: true }),
        Animated.timing(tagY, { toValue: 0, duration: 600, easing: Easing.bounce, useNativeDriver: true }),

        Animated.delay(500)
      ])
    ).start();

    Animated.loop(
```



```
frontend > src > screens > LoginScreen.tsx > s > tabWrap
const LoginScreen = ({ navigation }: any) => {
  useEffect(() => {
      Animated.sequence([
        Animated.delay(5000),
        Animated.timing(flipAnim, { toValue: 1, duration: 800, easing: Easing.inOut(Easing.cubic), useNativeDriver: true }),
        Animated.timing(flipAnim, { toValue: 0, duration: 0, useNativeDriver: true }),
        Animated.delay(700)
      ])
    ).start();

    const jump = (anim: Animated.Value) => Animated.sequence([
      Animated.timing(anim, { toValue: -12, duration: 250, easing: Easing.out(Easing.quad), useNativeDriver: true }),
      Animated.timing(anim, { toValue: 0, duration: 400, easing: Easing.bounce, useNativeDriver: true }),
      Animated.delay(1200)
    ]);

    Animated.loop(
      Animated.stagger(150, [jump(dot3Y), jump(dot2Y), jump(dot1Y)])
    ).start();

  }, [tagY, flipAnim, dot1Y, dot2Y, dot3Y]);

  const spinY = flipAnim.interpolate({ inputRange: [0, 1], outputRange: ['0deg', '360deg'] });

  const shadowScale = tagY.interpolate({
    inputRange: [-70, -18, 0],
    outputRange: [0.2, 0.6, 1.2],
    extrapolate: 'clamp'
  });
  const shadowOpacity = tagY.interpolate({
    inputRange: [-70, -18, 0],
    outputRange: [0.05, 0.2, 0.6],
    extrapolate: 'clamp'
  });

  const burstDistance = flipAnim.interpolate({ inputRange: [0, 1], outputRange: [0, 50] });
  const burstOpacity = flipAnim.interpolate({ inputRange: [0, 0.2, 0.8, 1], outputRange: [0, 1, 0.8, 0] });

  const handleSubmit = async () => {
    setError('');

    const API_URL = process.env.EXPO_PUBLIC_API_URL;

    if (!API_URL) {
      setError("API URL is missing in .env file!");
```

```
frontend > src > screens > LoginScreen.tsx > s > tabWrap
  const LoginScreen = ({ navigation }: any) => {
    const handleSubmit = async () => {
      }

      if (isLogin) {
        if (!phone || !password) {
          setError('Please enter phone and password.');
          return;
        }
      } else {
        if (!phone || !password || !confirmPass) {
          setError('Please fill out all fields.');
          return;
        }
        if (password !== confirmPass) {
          setError('Passwords do not match.');
          return;
        }
      }

      const endpoint = isLogin ? '/api/auth/login' : '/api/auth/register';

      try {
        const response = await fetch(`${API_URL}${endpoint}`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            phoneNumber: phone,
            password: password
          }),
        });

        const data = await response.json();

        if (response.ok) {
          console.log(` Success! User Data:`, data);
          await AsyncStorage.setItem('userId', data._id);
          await switchUser(data._id);
          navigation.replace('Splash');
        } else {
          setError(data.error || "Authentication failed");
        }
      } catch (err) {
        console.error("Network error:", err);
        setError("Server unreachable. Check your IP address!");
      }
    };
```



```
ontend > src > screens > LoginScreen.tsx > s > tabWrap
  const s = StyleSheet.create({
    trailContainer: {
      flexDirection: 'row',
      alignItems: 'center',
      marginRight: 10,
      gap: 10,
      zIndex: 2,
    },
    trailDot: {
      width: 8, height: 8, borderRadius: 4,
      backgroundColor: '#4ADE80', shadowColor: '#4ADE80',
      shadowOffset: { width: 0, height: 2 }, shadowOpacity: 0.8, shadowRadius: 6, elevation: 4,
    },

    tagShapeContainer: {
      width: 70, height: 40, flexDirection: 'row', alignItems: 'center', position: 'relative',
      shadowColor: '#F5D1B0', shadowOffset: { width: 0, height: 6 }, shadowOpacity: 0.4, shadowRadius: 10, elevation: 8, zIndex: 5
    },
    tagBody: { position: 'absolute', right: 0, width: 50, height: 40, backgroundColor: '#F5D1B0', borderTopRightRadius: 8, borderB
    tagTip: { position: 'absolute', left: 20 - 14, top: 20 - 14, width: 28, height: 28, backgroundColor: '#F5D1B0', transform: [{
    tagHole: { position: 'absolute', left: 10, top: 20 - 5, width: 10, height: 10, borderRadius: 5, backgroundColor: '#2D464C', zI

    brandName: { fontSize: 36, fontWeight: '900', color: '#FFFFFF', letterSpacing: -1 },
    subhead: { fontSize: 14, color: 'rgba(255,255,255,0.6)', fontWeight: '600', marginTop: 8 },

    form: { width: '100%' },
    tabWrap: {
      flexDirection: 'row',
      backgroundColor: 'rgba(255,255,255,0.06)',
      borderRadius: 12,
      padding: 4,
      marginBottom: 20
    },
    tab: {
      flex: 1,
      alignItems: 'center',
      paddingVertical: 12,
      borderRadius: 10
    },
    tabActive: { backgroundColor: '#F5D1B0' },
    tabText: { fontSize: 14, fontWeight: '700', color: 'rgba(255,255,255,0.5)' },
    tabTextActive: { color: '#2D464C' },

    errorTxt: { color: '#FF8484', textAlign: 'center', marginBottom: 15, fontWeight: '700' },
    inputWrap: { flexDirection: 'row', alignItems: 'center', backgroundColor: 'rgba(255,255,255,0.06)', borderRadius: 16, height:
    input: { flex: 1, marginLeft: 15, color: '#FFFFFF', fontSize: 16, fontWeight: '600' },
    loginBtn: { flexDirection: 'row', backgroundColor: '#F5D1B0', height: 60, borderRadius: 16, alignItems: 'center', justifyConte
    loginBtnTxt: { color: '#2D464C', fontSize: 18, fontWeight: '800' },
  });
```

iii) index.ts

frontend > TS index.ts

```ts
import { registerRootComponent } from 'expo';
import App from './App';

registerRootComponent(App);

```

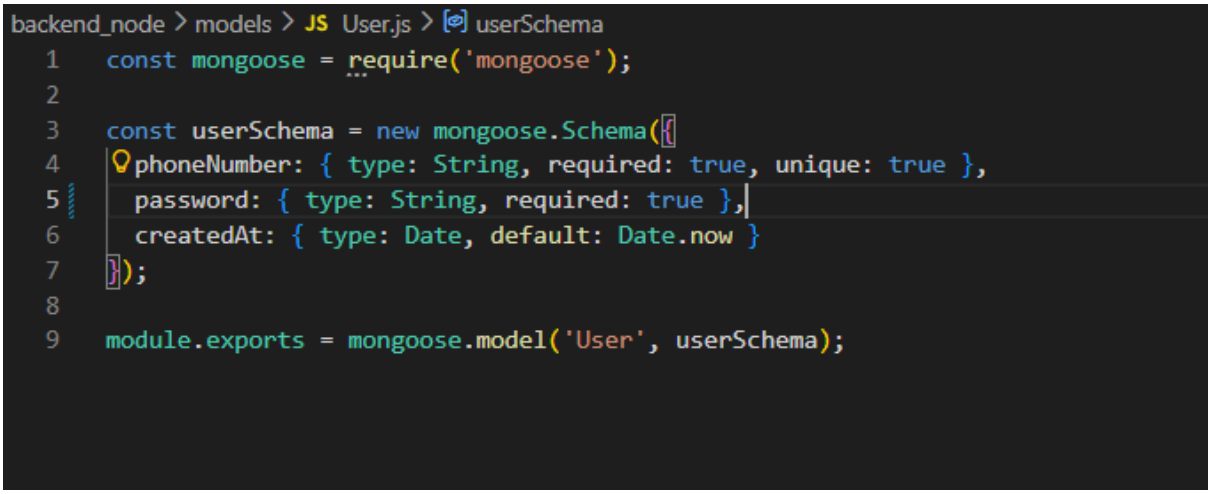
iv) AppNavigator.tsx

frontend > src > navigation > AppNavigator.tsx > default

```tsx
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createDrawerNavigator } from '@react-navigation/drawer';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import { DocumentProvider } from '../hooks/useDocuments';

import LoginScreen from '../screens/LoginScreen';

import DrawerContent from '../components/DrawerContent';
import DashboardScreen from '../screens/DashboardScreen';
import {
  PrivateScreen,
  PublicScreen,
  RestrictedScreen,
} from '../screens/CategoryScreens';
import {
  StatsScreen,
  LogsScreen,
  TrashScreen,
} from '../screens/OtherScreens';
import SplashScreen from '../screens/SplashScreen';
import { COLORS } from '../constants/theme';
import { RootStackParamList, DrawerParamList } from '../types/types';

const Drawer = createDrawerNavigator<DrawerParamList>();
const Stack  = createNativeStackNavigator<RootStackParamList>();

const DrawerNav: React.FC = () => (
  <Drawer.Navigator
    drawerContent={props => <DrawerContent {...props} />}
    screenOptions={{
      headerShown:    false,
      drawerStyle:    { width: '72%', backgroundColor: COLORS.background },
      overlayColor:   COLORS.overlay,
      swipeEdgeWidth: 60,
    }}
    initialRouteName="Dashboard"
  >
    <Drawer.Screen name="Dashboard"  component={DashboardScreen} />
    <Drawer.Screen name="Private"    component={PrivateScreen}   />
    <Drawer.Screen name="Public"     component={PublicScreen}    />
    <Drawer.Screen name="Restricted" component={RestrictedScreen} />
    <Drawer.Screen name="Trash"      component={TrashScreen}     />
    <Drawer.Screen name="Stats"      component={StatsScreen}     />
    <Drawer.Screen name="Logs"       component={LogsScreen}      />
  </Drawer.Navigator>
);

```

Backend_Node :

i) User.js



ii) Document.js

iii) Server.js

backend_node > JS server.js > ...

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const bcrypt = require('bcryptjs');
const http = require('http');
const { Server } = require('socket.io');

const fs = require('fs');
const multer = require('multer');
const axios = require('axios');
const cloudinary = require('cloudinary').v2;

const User = require('./models/User');
const Document = require('./models/Document');

const app = express();
const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: "*",
    methods: ["GET", "POST"]
  }
});

io.on('connection', (socket) => {
  console.log(`📡 Frontend connected to Loudspeaker: ${socket.id}`);

  socket.on('disconnect', () => {
    console.log(`🔴 Frontend disconnected: ${socket.id}`);
  });
});

app.use(cors());
app.use(express.json());

app.use((req, res, next) => {
  res.setHeader('ngrok-skip-browser-warning', 'true');
  next();
});

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
```

backend_node > JS server.js > ...

```
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});
const upload = multer({ dest: 'uploads/' });

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('✅ Connected to MongoDB Atlas!'))
  .catch((err) => console.error('❌ MongoDB connection failed:', err.message));

app.post('/api/auth/register', async (req, res) => {
  try {
    const { phoneNumber, password } = req.body;

    if (!phoneNumber || !password) {
      return res.status(400).json({ error: "Phone number and password are required" });
    }

    let user = await User.findOne({ phoneNumber });
    if (user) {
      return res.status(400).json({ error: "User already exists. Please login." });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    user = new User({ phoneNumber, password: hashedPassword });
    await user.save();

    console.log(`✨ New user created: ${phoneNumber}`);
    res.json({ _id: user._id, phoneNumber: user.phoneNumber });

  } catch (error) {
    res.status(500).json({ error: "Server error during registration" });
  }
});
```

backend_node > JS server.js > ...

```
205
206  app.delete('/api/documents/:docId', async (req, res) => {
207    try {
208      const { docId } = req.params;
209      await Document.findByIdAndDelete(docId);
210      res.json({ message: "Document permanently deleted" });
211    } catch (error) {
212      res.status(500).json({ error: "Failed to delete document" });
213    }
214  });
215
216  app.delete('/api/documents/empty-trash/:userId', async (req, res) => {
217    try {
218      const { userId } = req.params;
219      const result = await Document.deleteMany({ userId, category: 'Trash' });
220      res.json({ message: `Emptied ${result.deletedCount} documents from Trash` });
221    } catch (error) {
222      res.status(500).json({ error: "Failed to empty trash" });
223    }
224  });
225
226  app.post('/api/documents/manual-upload', upload.single('file'), async (req, res) => {
227    try {
228      const { userId, type, url } = req.body;
229      let finalUrl = url;
230
231      if (type === 'pdf' && req.file) {
232        console.log("📥 Catching PDF from App, uploading to Cloudinary...");
233
234        const cloudRes = await new Promise((resolve, reject) => {
235          cloudinary.uploader.upload_large(
236            req.file.path,
237            { resource_type: "raw" },
238            function(error, result) {
239              if (error) {
240                console.error("☁️ Cloudinary Upload Error:", error);
241                reject(error);
242              } else {
243                resolve(result);
244              }
245            }
246          );
247        });
```

backend_node > JS server.js > ...

```
226  app.post('/api/documents/manual-upload', upload.single('file'), async (req, res) => {
280          }
281
282          console.log(`😴 Python didn't answer (${axiosErr.code || axiosErr.message}). Waiting 8 seconds...`);
283          await new Promise(resolve => setTimeout(resolve, 8000));
284        }
285      }
286
287      console.log(`✅ Python replied with Status ${pythonRes.status}:`, pythonRes.data);
288      res.status(200).json({ message: "Sent to AI Engine successfully" });
289
290    } catch (error) {
291      console.error("❌ Manual upload failed:", error);
292      res.status(500).json({ error: "Server error during upload" });
293    }
294  });
295
296  app.post('/api/ai/webhook', (req, res) => {
297    try {
298      const { title, security_status, message } = req.body;
299
300      const liveLog = {
301        id: Date.now().toString(),
302        message: message || `AI Analysis complete for "${title}"`,
303        status: security_status === 'flagged' ? 'warning' : 'success',
304        timestamp: new Date().toLocaleTimeString()
305      };
306
307      console.log(`📣 Broadcasting to App: ${liveLog.message}`);
308
309      io.emit('AI_UPDATE', liveLog);
310
311      res.json({ success: true, message: "Node successfully shouted to the App!" });
312    } catch (error) {
313      console.error("Webhook error:", error);
314      res.status(500).json({ error: "Failed to broadcast signal" });
315    }
316  });
317
318  const PORT = process.env.PORT || 5000;
319  server.listen(PORT, '0.0.0.0', () => {
320    console.log(`🚀 TagAndTrail Backend (with WebSockets) running on port ${PORT}`);
321  });
```

Backend_python:

i) main.py

backend_python > main.py

```python
import os
import requests
import datetime
import json
from bson.objectid import ObjectId
from pymongo import MongoClient
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qs
import threading
import sys
import builtins
import logging

logging.basicConfig(
    stream=sys.stdout,
    level=logging.INFO,
    format='%(message)s'
)

def permanent_print(*args, **kwargs):
    logging.info(" ".join(map(str, args)))

builtins.print = permanent_print

from helpers import send_media_to_hf, send_text_to_hf
from config import TWILIO_SID, TWILIO_TOKEN, TAG_ENGINE_URL, TAG_ENGINE_KEY, MONGO_URI, NODE_SERVER_URL

client = MongoClient(MONGO_URI)
db = client.get_database('test')

def get_semantic_tags(title: str, text: str):
    print(f"\n🧠 [TAG ENGINE] Triggered! Sending Title: '{title}' | Text Length: {len(text)}")
    try:
        payload = {"title": title if title else "WhatsApp", "text": text}
        headers = {"Content-Type": "application/json", "X-API-Key": TAG_ENGINE_KEY}

        res = requests.post(TAG_ENGINE_URL, json=payload, headers=headers, timeout=15)
        print(f"🧠 [TAG ENGINE] HTTP Status Code: {res.status_code}")

        if res.status_code == 200:
            data = res.json()
            print(f"🧠 [TAG ENGINE] Raw JSON Response: {data}")

            tags = data.get("tags", [])
            print(f"🧠 [TAG ENGINE] Extracted Tags Array: {tags}\n")
            return tags
        else:
            print(f"❌ [TAG ENGINE] API Error Text: {res.text}\n")

    except Exception as e:
        print(f"❌ [TAG ENGINE] Crash/Timeout Exception: {str(e)}\n")

```

backend_python > main.py

```python

    def get_semantic_tags(title: str, text: str):
        print(f"\n🧠 [TAG ENGINE] Triggered! Sending Title: '{title}' | Text Length: {len(text)}")
        try:
            payload = {"title": title if title else "WhatsApp", "text": text}
            headers = {"Content-Type": "application/json", "X-API-Key": TAG_ENGINE_KEY}

            res = requests.post(TAG_ENGINE_URL, json=payload, headers=headers, timeout=15)
            print(f"🧠 [TAG ENGINE] HTTP Status Code: {res.status_code}")

            if res.status_code == 200:
                data = res.json()
                print(f"🧠 [TAG ENGINE] Raw JSON Response: {data}")

                tags = data.get("tags", [])
                print(f"🧠 [TAG ENGINE] Extracted Tags Array: {tags}\n")
                return tags
            else:
                print(f"❌ [TAG ENGINE] API Error Text: {res.text}\n")

        except Exception as e:
            print(f"❌ [TAG ENGINE] Crash/Timeout Exception: {str(e)}\n")

        return []

    def run_ai_pipeline(result, user):
        tag_title = ""
        tag_text = ""

        if "model_prediction" in result and isinstance(result["model_prediction"], dict):
            tag_title = result["model_prediction"].get("title", "")
            tag_text = result["model_prediction"].get("description", "")
        elif result.get("type") == "link" or "description" in result:
            tag_title = result.get("title", "")
            tag_text = result.get("description", "")
        elif result.get("type") == "text":
            tag_text = result.get("content", "")
            tag_title = tag_text[:30]

        print(f"🔎 [DEBUG PRE-TAG] Extracted tag_text: '{tag_text}' (Length: {len(tag_text) if tag_text else 0})")

        if tag_text and len(tag_text.strip()) > 5:
            tags = get_semantic_tags(tag_title, tag_text)
            if tags:
                result["semantic_tags"] = tags
        else:
            print("⚠️ [DEBUG] Skipped Tag Engine: tag_text was empty or too short!")

        print("\n--- Processing AI Result ---")
        print(result)
        print("----------------------\n")
```

backend_python > main.py

```python
    class WhatsAppHandler(BaseHTTPRequestHandler):
        def do_POST(self):
            try:
                self.end_headers()

            except Exception as e:
                print(f"❌ Global Server Crash: {str(e)}")
                self.send_response(500)
                self.end_headers()

    def classify_message(data):
        num_media = int(data.get("NumMedia", 0))
        text = data.get("Body", "")

        if num_media > 0:
            mime = data.get("MediaContentType0", "")
            url = data.get("MediaUrl0", "")

            if mime == "application/pdf":
                return send_media_to_hf(
                                url,
                                mime,
                                "pdf",
                                auth=(TWILIO_SID, TWILIO_TOKEN)
                            )
            else:
                if not TWILIO_SID or not TWILIO_TOKEN:
                    return {
                        "type": "media",
                        "original_mime": mime,
                        "url": url,
                        "error": "Missing Twilio credentials"
                    }

                return send_media_to_hf(
                    url,
                    mime,
                    "other_media",
                    auth=(TWILIO_SID, TWILIO_TOKEN)
                )

        return send_text_to_hf(text)


    def run():
        port = int(os.environ.get("PORT", 8000))
        server = HTTPServer(("0.0.0.0", port), WhatsAppHandler)
        print(f"☁️ Tag & Trail Python Server running on port {port}")
        server.serve_forever()

    if __name__ == "__main__":
        run()
```

backend_python > main.py

```python
def run_ai_pipeline(result, user):

    try:
        is_link = result.get("type") == "link"
        doc_type = "link" if is_link else "pdf"

        content_url = result.get("url") if is_link else result.get("file_url")

        doc_title = tag_title if tag_title else "Upload"
        if is_link and result.get("title"):
            doc_title = result.get("title")

        is_public = result.get("public", False)
        doc_category = "Public" if is_public else "Private"
        security_status = "safe"

        if result.get("prediction") in ["malicious", "phishing", "malware"]:
            doc_category = "Restricted"
            security_status = "flagged"

        if "model_prediction" in result and isinstance(result["model_prediction"], dict):
            pdf_risk = result["model_prediction"].get("risk_level", "").upper()
            pdf_pred = result["model_prediction"].get("prediction", "").lower()
            if pdf_risk in ["HIGH", "CRITICAL"] or pdf_pred in ["malicious", "malware"]:
                doc_category = "Restricted"
                security_status = "flagged"

        vt_data = result.get("virustotal") or result.get("virustotal_scan", {}).get("virustotal", {})
        if vt_data and vt_data.get("malicious", 0) > 0:
            doc_category = "Restricted"
            security_status = "flagged"

        print(f"🛡️ Security Status: {security_status.upper()} | Category: {doc_category}")

        new_document = {
            "userId": user["_id"],
            "title": doc_title,
            "type": doc_type,
            "category": doc_category,
            "security_status": security_status,
            "contentUrl": content_url if content_url else "No URL provided",
            "tags": result.get("semantic_tags", []),
            "metadata": result,
            "createdAt": datetime.datetime.now()
        }
```

backend_python > main.py

```python
def run_ai_pipeline(result, user):
    except Exception as e:


class WhatsAppHandler(BaseHTTPRequestHandler):
    def do_HEAD(self):
        self.send_response(200)
        self.end_headers()

    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-type', 'application/json')
        self.end_headers()
        self.wfile.write(json.dumps({"status": "awake", "message": "AI Engine is Live!"}).encode())

    def do_POST(self):
        try:
            content_length = int(self.headers.get('Content-Length', 0))
            body = self.rfile.read(content_length)

            if '/manual' in self.path:
                print("\n🚪 [DOOR 1] App Manual Upload Received!")
                try:
                    data = json.loads(body.decode('utf-8'))
                    print(f"📦 RAW Payload received from Node: {data}")

                    self.send_response(200)
                    self.send_header('Content-type', 'application/json')
                    self.end_headers()
                    self.wfile.write(json.dumps({"status": "received", "message": "Processing started"}).encode())

                    user_id = data.get("userId")
                    item_type = data.get("type")
                    url = data.get("url")

                    if not user_id:
                        print("⚠️ ERROR: No userId provided in the payload.")
                        return

                    print(f"🔍 Looking up user ID: {user_id}")
                    user = db.users.find_one({"_id": ObjectId(user_id)})

                    if not user:
                        print("⚠️ ERROR: User not found in DB.")
                        return

                    print("✅ User found! Processing AI in background...")

                    def background_worker():
                        try:
                            if item_type == "pdf":
                                result = send_media_to_hf(url, "application/pdf", "pdf", auth=None)
                            else:
                                result = send_text_to_hf(url)
```

ii) helpers.py

backend_python > helpers.py

```python
import os
import uuid
import requests
import re
import time
import socket
import cloudinary
import cloudinary.uploader

from urllib.parse import urlparse, unquote
from bs4 import BeautifulSoup
from PyPDF2 import PdfReader

from config import (
    DOWNLOAD_FOLDER, MAX_FILE_SIZE, VT_API_KEY, HF_API_URL,
    TWILIO_SID, TWILIO_TOKEN, CLOUDINARY_CLOUD_NAME,
    CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET
)

cloudinary.config(
  cloud_name = CLOUDINARY_CLOUD_NAME,
  api_key = CLOUDINARY_API_KEY,
  api_secret = CLOUDINARY_API_SECRET
)

def sanitize_filename(name: str) -> str:
    name = unquote(name)
    name = re.sub(r"[^a-zA-Z0-9._-]", "_", name)
    return name[:80] if name else "download"

def extract_filename(url, headers):
    cd = headers.get("Content-Disposition")
    if cd and "filename=" in cd:
        filename = cd.split("filename=")[-1].strip('"')
    else:
        filename = os.path.basename(urlparse(url).path)
    filename = sanitize_filename(filename)
    if not filename:
        filename = "download"
    name, ext = os.path.splitext(filename)
    if not ext:
        ext = ".pdf"
    return name, ext

def detect_file_type(path: str) -> str:
    with open(path, "rb") as f:
        header = f.read(16)
    if header.startswith(b"%PDF"): return "application/pdf"
    if header.startswith(b"\xff\xd8"): return "image/jpeg"
    if header.startswith(b"\x89PNG"): return "image/png"
    if header.startswith(b"GIF"): return "image/gif"
    if header.startswith(b"PK"): return "application/zip"
```

backend_python > helpers.py

```python
            is_encrypted = False
            if detected_mime == "application/pdf":
                if not is_pdf_public(file_path):
                    print("🔒 PDF is encrypted/password protected. Skipping AI text extraction.")
                    final_response["error"] = "PDF is encrypted/password protected."
                    final_response["public"] = False
                    is_encrypted = True

            print("🦠 Running VirusTotal scan...")
            final_response["virustotal_scan"] = scan_file(file_path)

            if not is_encrypted:
                print("🤗 Sending to Hugging Face AI...")
                with open(file_path, "rb") as f:
                    hf_response = requests.post(
                        f"{HF_API_URL}/predict_pdf",
                        files={"file": (base_name, f, detected_mime)},
                        timeout=60
                    )

                if hf_response.status_code == 200:
                    final_response["model_prediction"] = hf_response.json()
                else:
                    final_response["model_error"] = f"Hugging Face prediction failed: {hf_response.text}"
            else:
                final_response["model_prediction"] = {
                    "prediction": "encrypted",
                    "risk_level": "UNKNOWN",
                    "description": "File is password protected. AI scan bypassed."
                }

            if os.path.exists(file_path):
                try:
                    print(f"☁️ Uploading {base_name} to Cloudinary...")
                    upload_result = cloudinary.uploader.upload(file_path, resource_type="raw")

                    final_response["file_url"] = upload_result.get("secure_url")
                    print(f"✅ Uploaded successfully: {final_response.get('file_url')}")

                except Exception as e:
                    print(f"❌ Cloudinary upload failed: {str(e)}")
                    final_response["upload_error"] = f"Cloudinary upload failed: {str(e)}"

                finally:
                    os.remove(file_path)

            return final_response

        except Exception as e:
            return {"error": f"Media processing failed: {str(e)}"}
```

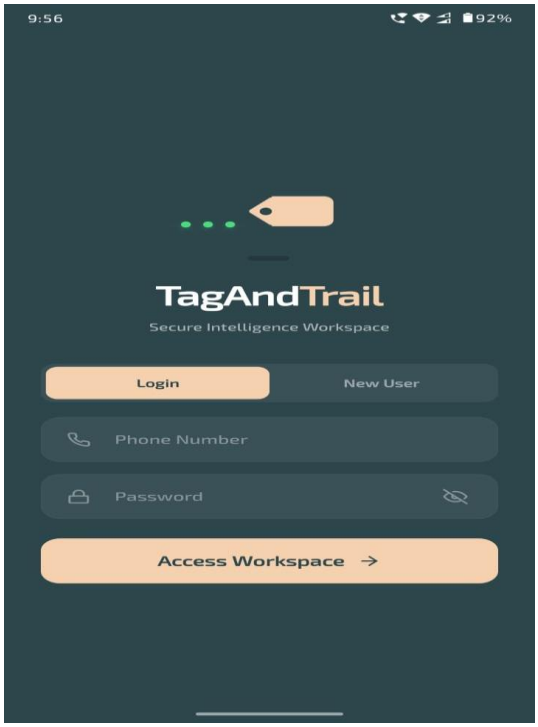
iii) requirements.txt

backend_python > requirements.txt

```
1  requests==2.32.5
2  pymongo==4.10.1
3  dnspython==2.8.0
4  cloudinary==1.44.1
5  beautifulsoup4==4.14.3
6  PyPDF2==3.0.1
7  python-dotenv==1.0.1
```

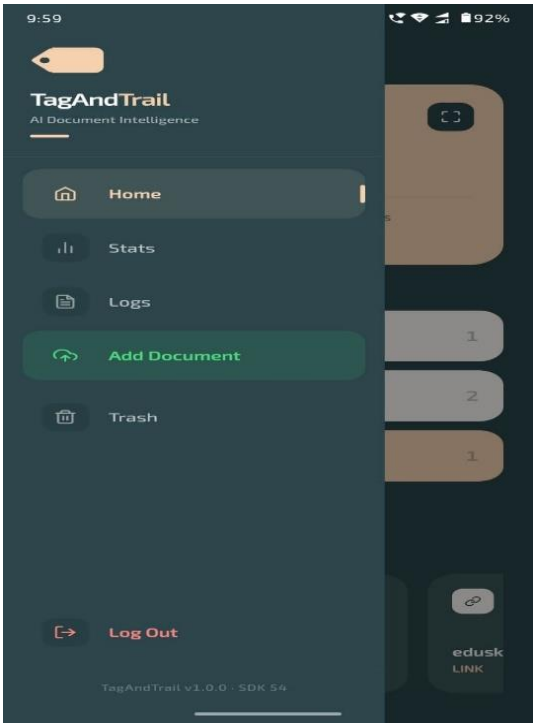
5.2 EXPERIMENTAL RESULTS

i) Login Screen



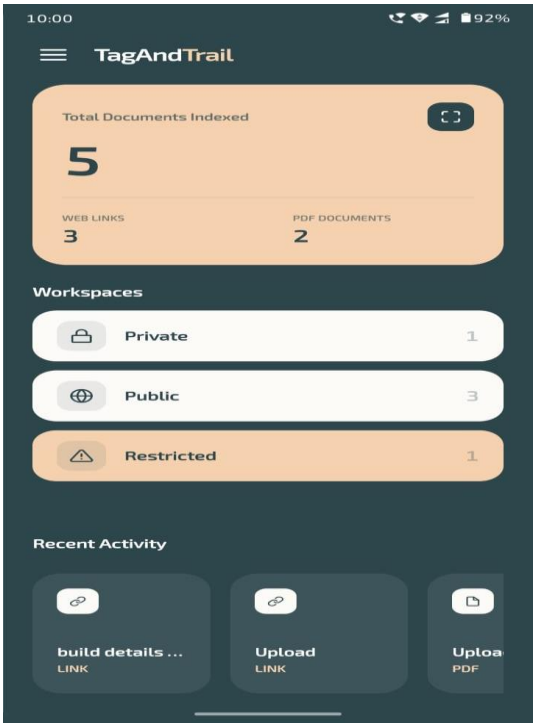
5.2.1 Login Screen

ii) Menu Section



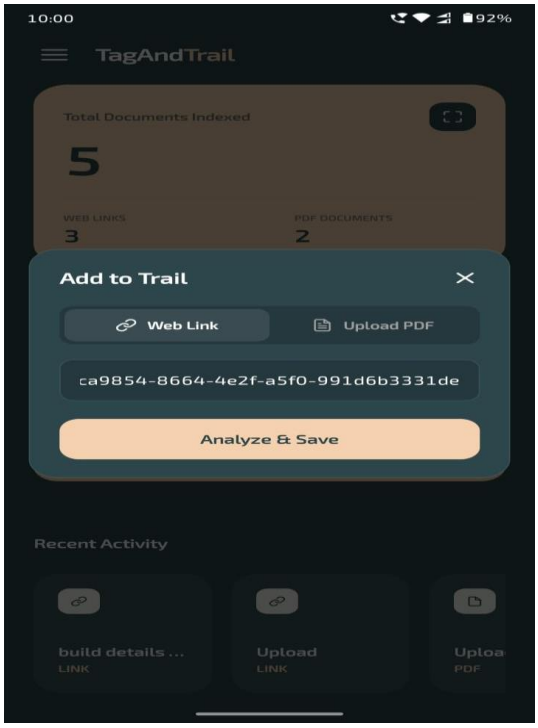
5.2.2 Menu Section

iii) DashBoard Screen



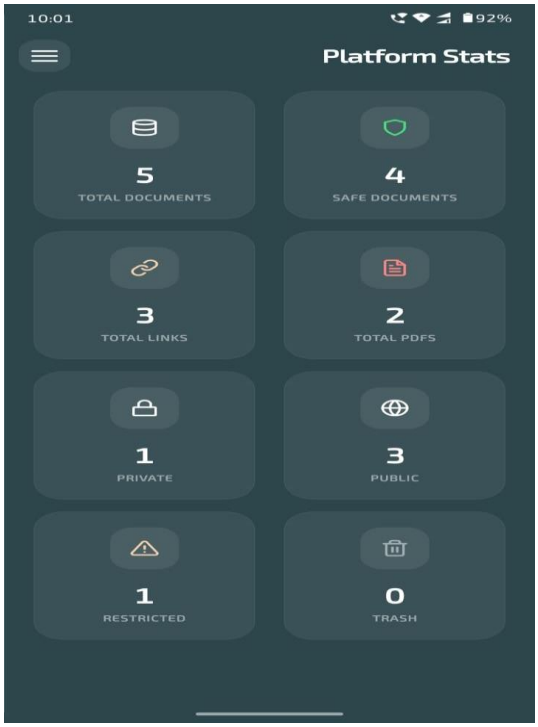
5.2.3 DashBoard Screen

iv) Upload Section



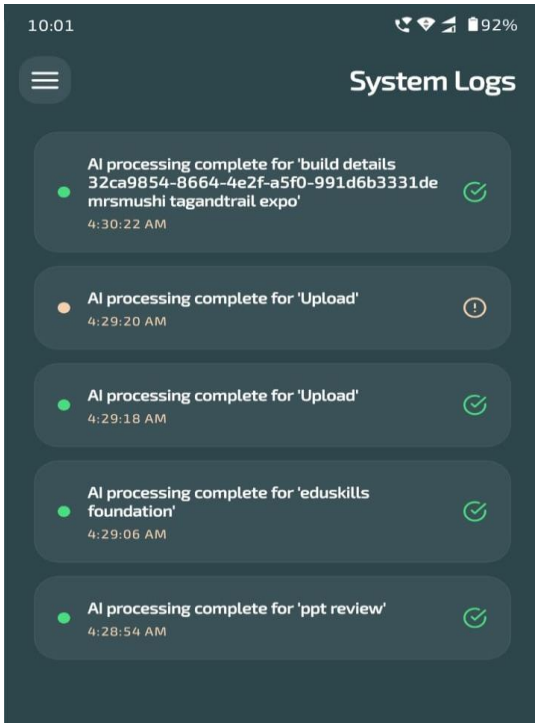
5.2.4 Upload Section

v) System Logs



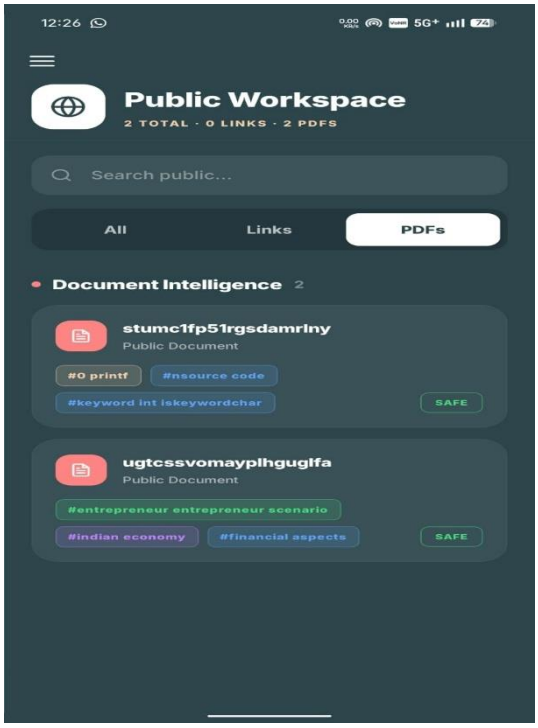
5.2.5 System Logs Section

vi) Stats Screen



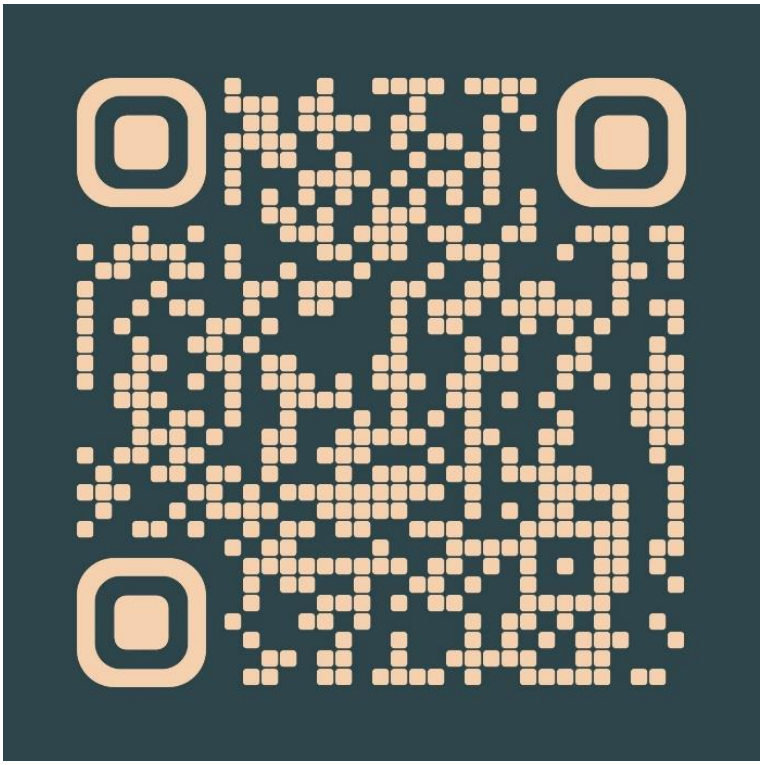
5.2.6 Stats Screen

vii) Uploaded Pdf Section



5.2.7 Uploaded Pdf Section

viii) QR code to Download the app



5.2.8 QR code to install the Tag And Trail App

5.3 TEST CASES

1. Authentication Module

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC01	Valid Login	Correct phone & password	User logged in successfully	Login successful	Pass
TC02	Invalid Login	Wrong password	Error message displayed	Error shown	Pass
TC03	Empty Fields	No input	Validation error	Error shown	Pass
TC04	User Registration	Valid details	Account created	Account created	Pass

Table - 5.3.1

2. Document / Link Input Module

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC05	Add Valid Link	Correct URL	Link processed	Link processed	Pass
TC06	Add Invalid Link	Incorrect URL	Error message	Error shown	Pass
TC07	Upload PDF	Valid PDF file	File uploaded	Upload success	Pass
TC08	Unsupported File	DOCX/Image	Error/Conversion	Handled	Pass
TC09	Empty Input	No data	Validation error	Error shown	Pass

Table - 5.3.2

3. Processing & Tag Generation

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC10	Tag Generation	Valid content	Relevant tags generated	Tags generated	Pass
TC11	Large Document	Long PDF	Tags generated	Success	Pass

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
			properly		
TC12	Empty Content	Blank input	Error / No tags	Handled	Pass
TC13	Keyword Extraction	Text content	Keywords extracted	Correct output	Pass

Table - 5.3.3

4. Classification Module

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC14	Public Classification	Open link	Classified as Public	Correct	Pass
TC15	Private Classification	Restricted link	Classified as Private	Correct	Pass
TC16	Restricted Content	Login-required link	Classified as Restricted	Correct	Pass

Table - 5.3.4

5. Search & Filter

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC21	Search Keyword	“edu”	Matching results displayed	Correct results	Pass
TC22	No Results	Random text	No results shown	Correct	Pass
TC23	Filter Links	Select Links	Only links displayed	Correct	Pass
TC24	Filter PDFs	Select PDFs	Only PDFs displayed	Correct	Pass

Table – 5.3.5

6. Safety Check Module

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Result
TC25	Safe Link	Valid URL	SAFE status	SAFE	Pass
TC26	Suspicious Link	Malicious URL	Warning displayed	Detected	Pass
TC27	PDF Safety	Uploaded file	Safety verified	Correct	Pass

Table - 5.3.6

CHAPTER-6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Tag And Trail system was developed in response to the increased significance of smart solutions to managing digital content in the current dynamically evolving world. As the number of communication platforms grows, and more links and documents are shared daily, users are overwhelmed with a lot of information. Nevertheless, the bulk of this information is disorganized and found in bits, thus hard to access and make good use of. Tag And Trail copes with this dilemma by introducing a system that does not only store the content but also makes it meaning and accessible.

The system has been made to deal with various kinds of inputs such as web links and PDF documents and process them in a systematic way. The system can comprehend the information and produce pertinent tags by using methods like content extraction, natural language processing and semantic analysis. These tags assist in sorting the information and simplify searching and accessing information by the user when required.

Automation of tasks which are otherwise carried out manually is one of the greatest feats of this project. Users do not have to navigate through various links or documents to find the information they seek, but the system can simplify this process by categorizing and displaying information in a simplified manner. This does not only save time, but it also minimizes the amount of effort that would be used to handle large amount of data. Safety checks are also included, which also guarantees that the system processes only trustworthy and safe material.

The successful implementation of several technologies is another significant project outcome. The system is made up of mobile application interface, back-end services, database management, and Artificial Intelligence (AI)-based processing modules. All the components collaborate to ensure a smooth and effective workflow, starting with data input to the end product. The testing stage will ensure that the system is consistent and competent to deal with various situations.

The project aims at enhancing user experience besides functionality. The interface is made easy to navigate and is simple and easy to interact with, enabling a user to interact with the system with ease. The ability to tag, categorize and search, are some of the features that improve the usability and increase the practicality of the system in day-to-day use.

As a whole, Tag And Trail manages to show how smart automation could enhance the manner in which individuals interact with digital information. It takes unstructured information and forms it into structured knowledge and offers a true solution to managing the growing amount of information in the digital world today.

6.2 Future Scope

The existing Tag And Trail is a good starting point towards creating an intelligent system of digital content management. Nonetheless, a great deal can be done to develop this system and enhance it in the field of real-life applications. Achieving more accurate and richer content comprehension is one of our primary objectives in the future as we would like to train our models using a broader range of data. This will enable the system to create more accurate tags and comprehend more various types of documents and links, no matter how intricate and wildly formatted they are.

We are also looking into how to extend the content that can be dealt with by the system. Currently it is primarily based on links and PDF files, although in the future it may also be possible to add pictures, audio files and even video files. This would render the system more adaptable and able to handle more information that users face on a daily basis.

The other direction we would like to pursue is to improve the interaction of the users with the system. As an illustration, automatic summaries of documents and links would assist users to have a quick understanding of the core content without having to read the whole document. We will also consider personalizing the system by learning the preferences of the users and providing them with suggestions of relevant content in the system, depending on user activity.

From a technical perspective, moving towards a cloud-based infrastructure is an important step. This would enable the system to accommodate more data and accommodate more users simultaneously without any problem in performance. It would also allow users to access their data without any inconvenience in various gadgets and the experience would be more comfortable.

We are also considering incorporating Tag And Trail with other applications like messaging applications or productivity applications. This would enable users to save and organize contents directly without having to move on and off various applications. Moreover, the explanation of tags and classifications should be made easier and more conversational, to enable users to have

a better understanding of how the system works with their data.

Finally, Tag And Trail vision is to be more than just a pure storage and transform itself into a smart knowledge management system. The idea is to minimize the effort needed to handle information and simplify the process of users accessing, comprehending and making good use of their information. We are very eager about the future and we are eager to keep on making the system better and better so as to add meaningful value to the users.

CHAPTER-7

REFERENCES

1. A Genetic-Algorithms Matrix-Factorization-Based Recommender System Using Tags (2025),
https://ieeexplore.ieee.org/

2. Extracting Sentence Embeddings from Pretrained Transformer Models (2025),
https://arxiv.org/

3. Web Crawling Algorithm Fusing TF-IDF and Word2Vec Feature Extraction (2025),
https://www.sciencedirect.com/

4. PDF Malware Detection: Toward Machine Learning Modeling With Explainability Analysis (2024),
https://ieeexplore.ieee.org/

5. Exploring the Landscape of Automatic Text Summarization: A Comprehensive Survey (2023),
https://arxiv.org/

6. A Retrieval-Augmented Generation Based Large Language Model Benchmarked on a Novel Dataset (2023),
https://arxiv.org/

7. SemGloVe: Semantic Co-occurrences for GloVe from BERT (2022),
https://aclanthology.org/

8. Unsupervised Keyphrase Extraction via Interpretable Neural Networks (2022), https://arxiv.org/abs/2203.07640/

9. PatternRank: Leveraging Pretrained Language Models and Part of Speech for Unsupervised Keyphrase Extraction (2022), https://arxiv.org/abs/2210.05245/

10. LongKey: Keyphrase Extraction for Long Documents (2024), https://arxiv.org/abs/2411.17863/